

Computer Graphics I

IGWT lecture notes, extra exercises, snippet catalog, and the shared software-rasterizer kernel. Live HTML demos stay in [Computer Graphics/code/](#).

Course plan

Computer Graphics I

A 15-week university course for the **Interactive Graphics and Web Technologies (IGWT)** program.

Students learn how an image is made: pixels, transforms, cameras, rasterization, lighting, and textures. They implement a **tiny software renderer** in JavaScript, then map every stage onto WebGL / Three.js so later courses are names they already understand.

Source of teaching format: 02 Curriculum Design Advice

Lecture notes (teach from these): 00 Lectures

Extra exercises: 00 Index

Code snippets / software rasterizer: 11 Computer Graphics Snippets · [Computer Graphics/code/index.html]

Related later courses (do not steal their labs):

- 07 WebGL and Shader Snippets — GPU API, Semester 3
- 08 Three.js Snippets — engine, Semester 3
- Real-Time Rendering / Shader Programming — PBR, shadows, post, Semester 4

Where this course sits

Teach it in **Semester 2**, after:

- 01 subjects#3. Mathematics for Computer Graphics
- 01 subjects#1. Introduction to Programming

Run it **alongside or just after** 04 Computational Geometry. Week 13 of computational geometry (BVH picking) and Week 13 of this course (GPU mapping) can share a joint session.

Prerequisite	Why it is required
Programming (JavaScript)	Every lab writes a renderer or a shader-sized function
Vectors, matrices, trigonometry	Transforms, cameras, lighting
Canvas or DOM comfort	Framebuffer is an <code>ImageData</code>

Do **not** start this course before students can write nested loops and small vector functions. Do **not** start with Three.js as the student algorithm. Three.js is a **named map of the pipeline** in Week 13, and an optional camera-move demo in Week 7, not the weekly lab.

Course goal

By the end of the semester, a student can:

1. Trace a vertex from object space to a pixel (model → world → view → clip → NDC → screen).
 2. Rasterize a triangle with barycentric coordinates and a depth buffer.
 3. Light a surface with Lambert and Blinn-Phong using interpolated normals.
 4. Sample a texture with UV coordinates and state the min/mag problem.
 5. Point at a WebGL / Three.js program and name which stage of **their** software renderer it is.
-

Teaching structure (every week)

Same five-part week as computational geometry.

Part	Duration	What happens
Lecture	75 min	Definitions, one derivation, one picture of the pipeline
Live coding	60 min	Professor implements that stage in the shared software renderer
Lab	2–3 hours	Students finish a starter and see pixels change
Homework	4–6 hours	One implementation + 3–5 written questions
Quiz	10 min	Spaces, matrices, barycentric, one picture question

Implementation language: JavaScript + Canvas `ImageData` for Weeks 1–12. **WebGL2** appears in Week 13 as the same scene on the GPU. Python is acceptable only if the department already standardized on it; then use NumPy arrays as the framebuffer.

Never lecture from slides only. Every week a picture on screen must change because of code written in class.

Conventions (freeze in Week 1, keep all year)

Copy the WebGL course freeze so students do not relearn handedness in Semester 3. Full list: 01 Conventions

```
object → world → view → clip → NDC → pixels  
p_clip = P * V * M * p_object
```

Choice	This course
Handedness	Right-handed
Up	+Y
Camera looks	−Z
Matrices	Column-major; multiply <code>P * V * M * vec4(p,1)</code>
Front face	Counter-clockwise
Color	Lighting in linear space; gamma 2.2 at the end (Week 11–12)
Angles	Radians in code; degrees only on the board if you convert in front of them

If a cube is inside-out, it is winding or a mirrored scale — not “the API is broken.”

Professor preparation (before Week 1)

Prepare these once; reuse them every year.

- Pipeline posters: one wall of spaces, one wall of a triangle going to pixels
- A course repository:

```
cg1/
lecture/
starter/
solutions/
assignments/
raster/      # shared software renderer (ImageData)
math/        # vec3, mat4, barycentric
tests/       # transform, barycentric, depth fixtures
```

- A shared **math kernel** students import (they may rewrite it; they may not treat it as magic):
 - `vec3`, `dot`, `cross`, `normalize`
 - `mat4` `multiply`, `translate`, `rotate`, `scale`, `lookAt`, `perspective`, `ortho`
 - `barycentric(p, a, b, c)`
 - Auto-grader tests for `mat4` `multiply` and `barycentric` (including a point on an edge)
 - One recorded 8-minute demo per week
 - A 12-triangle cube OBJ (or hardcoded) used from Week 6 onward so scenes stay comparable
-

Assessment

Component	Weight	Notes
Labs (12)	25%	Must write pixels or a shader-sized function
Homework (8)	20%	Mix of code and short derivations
Quizzes (10)	10%	Weekly, 10 minutes
Midterm (Week 8)	15%	Written: spaces, matrices, raster, no laptop
Final project	30%	Working renderer or scene + 6–8 page report

No exam-only course. The project is the proof of mastery.

15-week lecture plan

Week 1 — What computer graphics is

Goal

Students see that graphics is **geometry + sampling + shading**, and they learn the course rules: a pipeline of spaces, a framebuffer, and no magic engines yet.

Lecture

- What an image is: a 2D array of colors
- The real-time pipeline vs offline ray tracing (name both; implement rasterization)
- Object, camera, light, material, framebuffer
- Course map for 15 weeks
- Conventions: right-handed, Y-up, -Z look, CCW

Live coding

Create a canvas, get `ImageData`, write a nested loop that fills a gradient. Put the image. Show that `(0,0)` is **top-left in the canvas**, and that we will convert from a y-up world later.

Lab

Checkerboard; then a function `putPixel(x, y, rgb)` with bounds checks.

Homework

1. Written: pipeline boxes from object to pixel, one sentence each.
2. Code: `clear(color)` and a 100×100 red rectangle that does not wrap.

Quiz

Is the canvas origin the same as the world origin? Yes/no and one sentence.

Week 2 — Color, pixels, and the framebuffer

Goal

Students can talk about resolution, aspect ratio, alpha, and why 8-bit sRGB is not linear light.

Lecture

- Pixel as a sample, not a little square (teaching level)
- RGB, alpha, premultiplied vs straight
- Framebuffer, pitch, byte order
- Aspect ratio; letterboxing
- sRGB vs linear (name it; full gamma policy in Week 11)

Live coding

Blend two overlapping translucent rectangles. Show the bug if you interpolate 8-bit sRGB as if it were linear (bright gray in the middle of black–white).

Lab

`drawLine` stub using DDA (next week finishes Bresenham). Alpha blend `over`.

Homework

Written: why averaging two sRGB pixels is not the same as averaging linear radiance. One picture.

Week 3 — Rasterizing lines and triangles in 2D

Goal

A filled triangle on the canvas from three pixel coordinates. This is the heart of the course.

Lecture

- DDA vs Bresenham (idea)
- Triangle as three half-planes vs barycentric
- Edge equations and the top-left fill rule (why cracks appear otherwise)
- Bounding box of a triangle as a reject, like AABB in computational geometry

Live coding

Barycentric fill of one 2D triangle. Color by barycentric weights ($RGB = \alpha\beta\gamma$).

Lab

Mesh of two triangles (quad). Handle a degenerate (collinear) triangle without crashing.

Homework

Implement `barycentric`. Tests: centroid, vertex, outside, on edge.

Week 4 — Vectors, points, and frames

Goal

Points are not vectors. Bases change. This week is the math they thought they already knew, now tied to a renderer.

Lecture

- Point vs vector vs normal (normals transform differently — preview)
- Dot, cross, length, orthonormal bases
- Coordinate frames; change of basis
- Link to 04 Computational Geometry: `orient` is a 2D cross product. Same sign story in 3D as a scalar triple product later.

Live coding

Orbit a 2D point around the origin with `cos / sin`. Then do it with a 2×2 matrix. Same picture.

Lab

Unit tests for `dot`, `cross`, `normalize`. Visual: draw basis vectors after a rotation.

Homework

Written: why you must not add two points. Code: `lerp` and `slerp` are **not** this week; only lerp of points along a segment.

Week 5 — Homogeneous coordinates and affine transforms

Goal

4×4 matrices so translation and projection live in the same multiply.

Lecture

- Affine = linear + translation
- Homogeneous $(x, y, z, 1)$ vs directions $(x, y, z, 0)$
- Translate, rotate about X/Y/Z, scale
- Composition: **right to left** if column vectors ($T * R * p$ means rotate first)
- Inverse of T, R, S (and when scale is non-uniform)

Live coding

`mat4` multiply in JS. Animate translate ◦ rotate vs rotate ◦ translate. Students must see they differ.

Lab

Rotate a triangle about its **center**, not the origin. (Translate to origin, rotate, translate back.)

Homework

Multiply two matrices by hand (2×2 warmup, then one 4×4 with mostly zeros). Code: `rotateY`.

Week 6 — Hierarchical models and scene graphs

Goal

A solar system / robot arm from a tree of transforms. This is how glTF and Three.js scenes are stored.

Lecture

- Model matrix as object → world
- Parent × local
- Scene graph; traversal
- Instancing as the same mesh, different M
- Normal matrix teaser: `transpose(inverse(M))` if non-uniform scale

Live coding

Sun, Earth, Moon: three cubes or three triangles, parented. Time slider.

Lab

A two-bone arm or a turret on a tank. Draw local axes at each node (debug).

Homework

Written: give `M_world` for a node three levels deep. Code: scene graph with `children` and `localMatrix`.

Week 7 — Cameras and the view transform

Goal

`lookAt` and why the view matrix is the inverse of the camera's world transform.

Lecture

- Eye, target, up
- Camera basis: w (look), u , v
- View matrix V
- Moving the world vs moving the camera
- Optional 5-minute Three.js orbit demo as an **oracle** of the same spaces (08 Three.js Snippets), then back to student matrices

Live coding

Implement `lookAt`. Fly the camera around the Week 6 solar system.

Lab

WASD or sliders: eye position. A "look at origin" button.

Homework

Written: if the camera is at $(0,0,5)$ looking at origin, what is V on the diagonal (teaching numbers).

Code: `lookAt` tests.

Week 8 — Midterm and perspective projection

Midterm (60–75 min)

Written. Spaces, 2D barycentric, matrix order, point vs vector, scene-graph multiply. Topic list issued in Week 7.

Lecture (remainder)

- Orthographic vs perspective
- Frustum: fov, aspect, near, far
- Clip space and w
- Why we do **not** divide by w in the vertex shader later (GPU does it)

Live coding

`perspective(fov, aspect, near, far)`. Draw a cube that foreshortens.

Lab

Toggle ortho / perspective on the same scene.

Week 9 — Clip, NDC, viewport, depth buffer

Goal

A correct 3D triangle rasterizer: transform, divide, viewport, z-buffer.

Lecture

- Clip volume; why near-plane clipping exists (optional: lerp clip, not full Sutherland–Hodgman)
- NDC in $[-1, 1]$
- Viewport to pixels; **y-flip** because canvas y grows down
- Depth buffer; `less` vs `lequal`
- Z-fighting; never set near = 0

Live coding

Full path: cube vertices → PVM → NDC → pixels + z-buffer. Two overlapping triangles, correct occlusion.

Lab

Render the course cube with depth. Disable depth: show the bug. Paint z as grayscale debug.

Homework

Written: why near too small causes z-fighting. Code: viewport transform with y-flip.

Week 10 — Lighting I (Lambert)

Goal

A lit cube that is not “each face a random color.”

Lecture

- Vertex normals vs face normals
- Lambert: $\max(0, \mathbf{n} \cdot \mathbf{l})$
- Ambient as a policy against pure black
- Light in **world** or **view** space — pick one and keep it
- Back-face culling vs lighting (they are not the same)

Live coding

Per-face Lambert on the cube. Move the light.

Lab

Per-vertex Lambert with barycentric interpolation of intensity (Gouraud). Compare to flat.

Homework

Written: why a scaled non-uniform mesh needs a normal matrix. Code: `transformNormal`.

Week 11 — Lighting II (Blinn-Phong) and color

Goal

Specular highlights and a gamma-aware output.

Lecture

- Phong vs Blinn-Phong ($h = \text{normalize}(l+v)$)
- Shininess; energy is not conserved (say it; PBR is Semester 4)
- Multiple lights: sum, then clamp or HDR teaser
- Linear vs sRGB: do lighting in linear; `pow(c, 1/2.2)` at the end

Live coding

Blinn-Phong on the cube. Toggle gamma.

Lab

Two lights. Debug view: normals as color (`n*0.5+0.5`).

Homework

Written: half-vector vs reflection vector. Code: `blinnPhong(n, l, v, kd, ks, shininess)`.

Week 12 — Texture mapping

Goal

A textured quad and a textured cube. UVs are just another interpolant.

Lecture

- UV in $[0,1]$; wrapping, clamping

- Perspective-correct interpolation (u/z , $1/z$) — teaching picture; implement if time
- Mag filter: nearest vs bilinear
- Min filter / mipmaps: name them; implement nearest for the lab
- Texture as albedo; still multiply by Lambert

Live coding

Load an image into an array. Sample nearest. Textured quad facing the camera, then on the cube.

Lab

Checkerboard procedural texture if image load is blocked (`file://`). UV debug coloring.

Homework

Written: why affine UV interpolation looks wrong on a perspective quad. Code: `sampleNearest(tex, u, v)`.

Week 13 — The same scene on the GPU

Goal

Students map **their** renderer onto WebGL2 (and optionally Three.js) without abandoning what they wrote.

Lecture

- Vertex shader = transform to clip
- Fragment shader = lighting + texture
- Attributes, uniforms, buffers
- Depth test, cull face
- Where Three.js hides V and P (`camera.matrixWorldInverse`, `projectionMatrix`)
- Joint note with computational geometry: picking is ray vs triangle, not "the engine knows"

Live coding

Port the course cube: one WebGL2 program, Lambert, one texture. Use `index.html` as the triangle-through-cube sequence if students are stuck on boilerplate — they still must explain each uniform.

Lab

Project checkpoint. Software **or** WebGL vertical slice: mesh + camera + light + texture + depth.

Homework

Project only. Written: a table "my CPU function → GPU stage."

Week 14 — Project studio

No new theory. Desk review: kernel (`mat4`, barycentric, z-buffer or shaders), visualizer, one ugly mesh, README.

Scope cuts

If they are behind	Cut
PBR / shadows	Lambert + one texture
Full glTF loader	Hardcoded cube + one OBJ
Deferred / post	Forward software or one WebGL pass
Beautiful UI	Canvas + keyboard
Three.js scene they cannot explain	Back to software rasterizer

A correct z-buffer on a plain canvas beats a broken `gltf` dump.

Week 15 — Project presentations

12 minutes + 5 minutes questions. Live demo, repo, 6–8 page report, 30-second recording.

Final project (choose one)

Every project must implement **the pipeline or a stage of it**, not only call Three.js. A library is allowed for windowing and image decode, not for the student claim (“I wrote a renderer”).

Project	Must include	Why it fits IGWT
Software mesh viewer	PVM, raster, z-buffer, Lambert, texture	Core course
Solar system / robot	Scene graph + camera	Animation, configurators
First-person room	lookAt, collision AABB optional	Games, XR later
OBJ + MTL viewer	Normals, UVs, multiple materials	Asset pipeline
CPU → GPU port	Same scene in software and WebGL	Bridge to Semester 3
2D UI renderer	Transforms + textured quads, no 3D	Weaker; allow only if 3D math is already solid

Project rubric

Criterion	Weight
Correct pipeline (tests + one debug view)	30%
Visual explanation of a stage	20%
Code quality and repository	15%
Report (spaces, limitations, citations)	20%
Presentation and live demo	15%

Week-by-week summary

Week	Topic	Students implement	Later-course payoff
1	Images, pipeline	<code>putPixel</code> , gradient	All labs
2	Color, framebuffer	Alpha over	Compositing
3	2D raster	Barycentric triangle	GPU fragments
4	Vectors and frames	<code>dot / cross</code>	Lighting
5	Homogeneous / affine	<code>mat4</code> , rotate-about-center	Every engine
6	Scene graph	Parent $\times$ local	glTF, Three.js
7	Camera	<code>lookAt</code>	Orbit / FPS
8	Midterm + projection	<code>perspective</code>	Clip space
9	NDC, viewport, depth	z-buffer cube	Hidden surface
10	Lambert	<code>n · l</code>	Materials
11	Blinn-Phong, gamma	Specular + sRGB out	PBR later
12	Textures	UV sample	glTF albedo
13	WebGL map	Same cube on GPU	Semester 3
14	Studio	Project	Portfolio
15	Presentations	Demo + report	Graduation evidence

Recommended textbooks

Assign **one** primary book. Use the others as references.

Book / site	Use
Marschner & Shirley — <i>Fundamentals of Computer Graphics</i>	Primary theory
Scratchapixel (rasterization, barycentric, cameras)	Free weekly companion
Angel & Shreiner — <i>Interactive Computer Graphics</i>	OpenGL-shaped pipeline
Hughes et al. — <i>Computer Graphics: Principles and Practice</i>	Reference depth
Akenine-Möller et al. — <i>Real-Time Rendering</i>	Not the undergraduate primary; steal pictures for Week 13+

Do not assign a Vulkan spec. Do not assign *Physically Based Rendering* as required reading in Semester 2.

What to skip in a first undergraduate course

Say this explicitly in Week 1 so the course does not balloon.

- Full perspective-correct interpolation proof (picture + optional lab extra)
- Full frustum clipping of triangles (clip against near, or drop triangles that straddle near)
- Shadow maps, SSAO, deferred, IBL, HDR pipelines
- Write-your-own GPU / WebGPU (Semester 4–5)
- Global illumination
- Three.js as the weekly implementation

Mention them. Do not grade them.

Faculty checklist for the first offering

1. Write `vec3/mat4/barycentric` and a one-triangle rasterizer in Week -2.
2. Record one 8-minute demo for Weeks 1–9 before the semester starts.
3. Prepare 10 quiz PDFs and the midterm.
4. Publish 3 project starter repos (software viewer, scene graph, WebGL cube).
5. Schedule Week 13 as a joint session with Computational Geometry Week 13 if both run.
6. Collect every student project into the annual IGWT exhibition.
7. Tell students where and live — **after** they can put a triangle in a framebuffer.

One-sentence teaching principle

Students should leave this course able to **look at a wrong picture, name the pipeline stage that failed, and write a debug view for it** (normals, UVs, depth, NDC).

Snippet catalog

Computer Graphics I snippets

Runnable software-rasterizer demos and a shared math kernel for IGWT Computer Graphics I.

Open: [Computer Graphics/code/index.html]

Copy from: [Computer Graphics/code/kernel.js]

Extra problem sets: [Computer Graphics/exercises/00 Index.md]

Language is **JavaScript + Canvas** `ImageData`. Do not require Three.js for weeks 1–12. WebGL2 cube is demo 16 (Week 13). Three.js may be a 5-minute camera oracle in Week 7, not the student `lookAt`.

If `file://` is blocked, from `Computer Graphics/code/`:

```
npx serve  
python -m http.server
```


Demos

#	File	Week	What it shows
1	[01-putpixel.html]	1	putPixel, gradient
2	[02-checker.html]	2	framebuffer
3	[03-alpha-over.html]	2	compositing
4	[04-barycentric.html]	3	$\alpha\beta\gamma$ as RGB
5	[05-quad.html]	3	two triangles
6	[06-vec3.html]	4	frames
7	[07-mat4-order.html]	5	T R vs R T
8	[08-rotate-center.html]	5	$T(c) R T(-c)$
9	[09-scene-graph.html]	6	parent $\times$ local
10	[10-lookat.html]	7	$V * eye \approx 0$
11	[11-perspective.html]	8	fov / ortho
12	[12-zbuffer.html]	9	depth
13	[13-lambert.html]	10	$n \cdot l$
14	[14-blinn-phong.html]	11	spec + gamma
15	[15-texture.html]	12	UV nearest
16	[16-webgl-cube.html]	13	same PVM on GPU
17	[17-kernel-tests.html]	all	fixtures
18	[18-project-sandbox.html]	14	debug views

Kernel (copy; do not grow fov until the cube “shows up”)

```
p_clip = P * V * M * vec4(p, 1)
ndc = clip.xyz / clip.w
sx = (ndc.x * 0.5 + 0.5) * width
sy = (1 - (ndc.y * 0.5 + 0.5)) * height // canvas y grows down
```

Full `vec3`, `mat4`, `lookAt`, `perspective`, `barycentric fill`, `z-buffer`, `Lambert`, `Blinn-Phong`, and `sampleNearest` are in `kernel.js`.

Lecture index

Computer Graphics I — Lecture notes

Parent plan: 10 Computer Graphics I

Teach these in order. Each note is one 75-minute lecture plus the live-coding, lab, homework, and quiz for that week.

Conventions for the whole term: 01 Conventions (right-handed, Y-up, camera looks $-Z$, $P * V * M$, CCW front).

Week	Note
1	Week 01 What Computer Graphics Is
2	Week 02 Color and Framebuffer
3	Week 03 Lines and Triangles
4	Week 04 Vectors and Frames
5	Week 05 Homogeneous Transforms
6	Week 06 Scene Graphs
7	Week 07 Cameras
8	Week 08 Midterm and Projection
9	Week 09 Clip NDC Depth
10	Week 10 Lambert Lighting
11	Week 11 Blinn Phong and Gamma
12	Week 12 Texture Mapping
13	Week 13 GPU Mapping
14	Week 14 Project Studio
15	Week 15 Presentations

Software renderer lives in [code/index.html] (`kernel.js`: `putPixel`, `mat4`, `lookAt`, `z-buffer`, `Lambert`, `texture`). WebGL and Three.js catalogs are **Semester 3** tools; use them as oracles in Weeks 7 and 13 only.

Extra exercises: 00 Index

Snippet catalog: 11 Computer Graphics Snippets

Week 01 What Computer Graphics Is

Week 1 — What computer graphics is

Time: 75 min lecture + 60 min live coding

Kernel this week: `putPixel(x, y, rgb)` on `ImageData`

Board first: object → world → view → clip → NDC → pixels

Timing

Minutes	Do this
0–10	Why this course exists (a picture is an array)
10–25	Rasterization vs ray tracing
25–45	Pipeline of spaces; who does what
45–60	Conventions; what we will not implement
60–70	Course contract, assessment, 15-week map
70–75	Preview <code>putPixel</code> , then stand up for live coding

Learning goals

1. Define an image as a 2D array of colors plus a framebuffer.
 2. Name rasterization and ray tracing and say which one this course implements.
 3. Recite the space chain: object → world → view → clip → NDC → pixels.
 4. State the course conventions (handedness, up, look, matrix order, winding).
 5. Write a pixel without wrapping the buffer.
-

1. Opening (10 min)

A computer has no idea what a “cube” is. It has numbers: positions, indices, colors. Graphics is the craft of turning those numbers into an array of pixels fast enough to animate.

Write:

```
geometry + camera + light + material → framebuffer
```

This program (IGWT) will later put that framebuffer on a website, a configurator, and XR. This semester we **build the arrow**.

Computational geometry (if they are in it) answers “which triangle? does it hit?” This course answers “what color is this pixel?”

2. Two ways to make a picture (15 min)

Rasterization (this course)

For each triangle:

1. Transform vertices to screen.
2. Find which pixels the triangle covers.
3. Shade those pixels (depth, light, texture).

The GPU does this. We will do it in JavaScript first so the GPU is not a shrine.

Ray tracing (name only)

For each pixel, shoot a ray, find the nearest surface, shade. Offline films; also hardware RT later. We do **not** implement a path tracer here.

Draw both. Students should be able to say: games and WebGL are rasterization-first.

3. The pipeline of spaces (20 min)

Draw six boxes. Fill them slowly.

Space	Question
Object	Where is this vertex on the mesh?
World	Where is it in the scene?
View	Where is it relative to the camera?
Clip	After projection, before divide; w matters
NDC	After divide; xyz typically in $[-1, 1]$
Pixels	Which framebuffer index?

The product they will write by Week 9:

```
p_clip = P * V * M * vec4(p_obj, 1)
p_ndc  = p_clip.xyz / p_clip.w
p_pix  = viewport(p_ndc)
```

Do not derive P today. Promise Week 8.

Who runs which box later

- CPU / your JS: build M, V, P, submit vertices
- Vertex shader: object → clip
- Rasterizer (fixed): clip → fragments
- Fragment shader: color

Week 13 is this table with WebGL names.

4. Conventions and skip list (15 min)

Freeze 01 Conventions now.

- Right-handed, +Y up, camera looks **-Z**
- Column vectors: `P * V * M * p`
- CCW winding is front
- Radians in code

We will not implement this term: shadow maps, deferred, PBR, Vulkan, a glTF parser as the core claim, Three.js as the weekly lab.

Three.js may appear for **five minutes** in Week 7 as a camera oracle. Students still write `lookAt`.

5. Course contract (10 min)

Assessment

Labs 25%, homework 20%, quizzes 10%, midterm 15%, project 30%.

How a week works

Lecture → live coding on a canvas → lab → homework → 10-minute quiz.

Language

JavaScript + Canvas `ImageData`. WebGL2 in Week 13.

Live coding (60 min)

Must work by the end of class:

1. Canvas in the DOM, backing store set (not 0×0).
2. `getImageData` / `putImageData` each frame (or after a draw).
3. `putPixel(x, y, r, g, b)` with integer coordinates and **bounds checks**.
4. A vertical gradient (y maps to intensity).

5. Print: width, height, `data.length === width * height * 4`.

Talk while typing:

- "Canvas y grows **down**. World y will grow **up**. We will flip in the viewport, Week 9, not today."
- "Index is `(y * width + x) * 4`. Off-by-one here is a black screen later."
- "I do not call `fillRect` for the renderer. `fillRect` is a UI overlay."

Starter they leave with:

```
export function putPixel(img, x, y, r, g, b, a = 255) {
  x = x | 0; y = y | 0;
  if (x < 0 || y < 0 || x >= img.width || y >= img.height) return;
  const i = (y * img.width + x) * 4;
  img.data[i] = r;
  img.data[i + 1] = g;
  img.data[i + 2] = b;
  img.data[i + 3] = a;
}
```

Lab

1. `clear(img, r, g, b)`.
2. Checkerboard with cell size 16.
3. Filled axis-aligned rectangle function that **clips** to the canvas.
4. Do not wrap coordinates (`x % width` is a bug unless you are writing a torus demo).

Done when a TA can resize the canvas and the checkerboard still tiles without throwing.

Homework

Due start of Week 2.

1. Pipeline diagram, six boxes, one sentence each.
 2. `putPixel` + `clear` + rectangle, 8 tests (corners, off-canvas, 1×1).
 3. Written: why we store 4 bytes per pixel. What is alpha **this week**? (We will blend in Week 2.)
-

Quiz (10 min)

1. Rasterization vs ray tracing: which loops over triangles first? (2 pts)
 2. Name the six spaces in order. (3 pts)
 3. Matrix product in this course: `P * V * M` or `M * V * P`? (2 pts)
 4. Canvas `(0,0)` is which corner? (3 pts)
-

Common mistakes

- Setting CSS size and forgetting `canvas.width / canvas.height`.
 - Using `fillRect` and calling it a renderer.
 - Assuming world +Y is canvas +Y.
 - Starting in Three.js "because the professor opened it once."
-

Board drawings

1. Six-space pipeline.
2. Framebuffer as a grid; one pixel's RGBA.
3. The 15-week map as five boxes: pixels, transforms, camera, shade, GPU.

Week 02 Color and Framebuffer

Week 2 — Color, pixels, and the framebuffer

Time: 75 min lecture + 60 min live coding

Kernel this week: alpha over; sRGB vs linear as a **named** problem

Board first: two overlapping squares, straight alpha vs premultiplied

Timing

Minutes	Do this
0–10	Quiz Week 1
10–25	Pixel as sample; resolution; aspect
25–45	RGB, alpha, compositing
45–60	sRGB vs linear (picture, not a full color-science course)
60–75	Aspect ratio, letterbox, framebuffer layout

Learning goals

1. Explain resolution vs CSS size vs framebuffer size.
 2. Composite with **over** (Porter–Duff) at teaching level.
 3. Say why 8-bit sRGB averages look too dark or too bright.
 4. Compute aspect ratio and why a circle becomes an ellipse.
 5. State byte order in `ImageData` (RGBA, 0–255).
-

1. Pixels and samples (15 min)

A pixel in this course is a **sample of color** at a grid location. Treating it as a little square is fine for raster rules; it is not a physical CRT phosphor.

Resolution: number of samples.

Density: samples per inch (mention, do not grade).

Aspect: `width / height` of the **framebuffer**, not of the window unless they match.

If the canvas is stretched with CSS, circles become ellipses. Fix: match backing store to CSS * `devicePixelRatio` (cap at 2), or letterbox.

2. Color and alpha (20 min)

`ImageData`: four uint8s, **RGBA**, unpadded rows.

Alpha 255 = opaque. Alpha 0 = invisible. This week we blend.

Over (straight, teaching):

```
out_rgb = src_rgb * src_a + dst_rgb * (1 - src_a)
out_a   = src_a + dst_a * (1 - src_a)
```

Work in 0–1, then convert to bytes. Premultiplied alpha is what GPUs often store; mention it; implement straight over in the lab.

Coverage vs transparency: a rasterized edge is a form of coverage. We will not do MSAA this term. A 50% alpha edge is a cheap fake.

3. sRGB vs linear (15 min)

Displays apply a curve. PNG and Canvas are typically **sRGB**. Light adds in **linear** intensity.

Demo they must see: blend black and white 50/50 in byte space → 128. That is not perceptually halfway, and it is not physically halfway.

Course policy:

- Weeks 2–10: store 8-bit as-is; do not pretend it is physically correct.
 - Weeks 11–12: convert to linear for lighting, gamma-encode at the end.
 - Never “fix” lighting by raising shininess until it looks Instagram.
-

4. Layout (10 min)

Index: $(y * \text{width} + x) * 4$.

No row padding in `ImageData`.

Clear color should be **visible** (0.1 gray), never (0,0,0) while debugging a black cube.

Live coding (60 min)

1. Two overlapping translucent rectangles, `over`.
2. Slider for source alpha.
3. Side-by-side: sRGB 50% gray vs a labeled “this is not linear 0.5.”
4. Letterbox helper: given window and framebuffer aspect, compute a centered viewport rectangle (used again in Week 9).

Lab

1. `overPixel(img, x, y, rgba)`.
2. Draw 8 translucent circles (or boxes) in a row with alpha 1/8, 2/8, ...
3. A "wrong CSS stretch" checkbox that sets CSS size $\neq$ backing store, and a caption.

Done when over of opaque red on blue is red, and over of alpha 0 does not change the destination.

Homework

1. Implement `over` with tests: opaque, invisible, 50% red on black.
 2. Written: one paragraph on sRGB vs linear. No need for the full 2.4 gamma formula; `pow(c, 2.2)` is enough.
 3. Written: a 16:9 framebuffer in a 4:3 window. Do you stretch or letterbox? Why?
-

Quiz (10 min)

1. `ImageData` index of pixel (2, 1) in a width-10 image? (2 pts)
 2. Write `over` for rgb assuming a in [0,1], ignore output alpha. (4 pts)
 3. Why does a CSS-stretched canvas distort a circle? (2 pts)
 4. Is byte 128 half as much light as 255? Yes/no. (2 pts)
-

Common mistakes

- Blending in uint8 without converting to 0–1.
 - Premultiplied and straight formulas mixed.
 - `clear` with alpha 0 then wondering why the page shows through.
 - Using `getContext('2d', { alpha: false })` and then debugging alpha.
-

Board drawings

1. Two squares, over.
2. sRGB curve sketch (not to scale; labeled).
3. Letterbox vs stretch.

Week 03 Lines and Triangles

Week 3 — Rasterizing lines and triangles in 2D

Time: 75 min lecture + 60 min live coding

Kernel this week: barycentric, filled triangle

Board first: three points, a pixel inside, weights α , β , γ

Timing

Minutes	Do this
0–10	Quiz Week 2
10–25	Lines: DDA, Bresenham idea
25–55	Triangles: half-planes vs barycentric
55–65	Fill rule and cracks
65–75	Bounding box as reject (link to CG AABB)

Learning goals

1. Rasterize a line without gaps (DDA is enough; Bresenham named).
 2. Compute barycentric coordinates of a pixel wrt a triangle.
 3. Fill a triangle using α , β , $\gamma \geq 0$ and $\alpha + \beta + \gamma = 1$ (with epsilon).
 4. Explain a top-left fill rule in one sentence.
 5. Interpolate a color (or any attribute) with barycentric weights.
-

1. Lines (15 min)

DDA: walk the longer axis, step the other by dy/dx . Round to pixels.

Bresenham: integer error term; mention; do not require the derivation.

Clip lines to the canvas (Cohen–Sutherland name only, or just reject endpoints outside and accept ugly clipped segments this week).

A line is **not** a thin triangle. Do not rasterize lines as triangles until they ask.

2. Barycentric coordinates (30 min)

Point p in triangle abc:

$$\begin{aligned} p &= \alpha a + \beta b + \gamma c \\ \alpha + \beta + \gamma &= 1 \\ \alpha, \beta, \gamma \geq 0 &\Leftrightarrow p \text{ inside (including boundary, policy)} \end{aligned}$$

Areas (signed):

$$\begin{aligned} \alpha &= \text{area}(pbc) / \text{area}(abc) \\ \beta &= \text{area}(pca) / \text{area}(abc) \\ \gamma &= \text{area}(pab) / \text{area}(abc) \end{aligned}$$

Signed area in 2D is the same cross product as Week 01 What Computational Geometry Is:

$$\text{area}(abc) = 0.5 * \text{cross}(b-a, c-a)$$

If $\text{area}(abc) = 0$, the triangle is degenerate: skip it.

Inside test: α, β, γ all $\geq -\text{eps}$ and $|\alpha + \beta + \gamma - 1|$ small (they should already sum to 1 algebraically).

Interpolation:

$$\text{color}(p) = \alpha \text{color}(a) + \beta \text{color}(b) + \gamma \text{color}(c)$$

Same formula for UV, normals, z (later).

3. Fill rule (10 min)

Shared edges would be drawn twice (or leave holes) if both triangles claim the boundary.

Top-left rule (teaching): a pixel on an edge belongs to the triangle that owns the top edge or the left edge. Implement a simple version: $\alpha \geq 0, \beta \geq 0, \gamma > 0$ (strict on one weight) **or** document "we double-draw shared edges; z-buffer will hide it in 3D."

Cracks = inconsistent edge tests or integer vs float pixel centers. Test at **pixel centers** $(x+0.5, y+0.5)$.

4. Bounding box (10 min)

Loop x from $\text{floor}(\min(ax, bx, cx))$ to $\text{ceil}(\max(\dots))$, same for y. Clip to canvas. Same idea as AABB reject in computational geometry: the box is not the triangle.

Live coding (60 min)

1. One triangle, vertices draggable (mouse in pixel space).
2. Fill with barycentric; $RGB = (\alpha, \beta, \gamma) * 255$.
3. Draw the bounding box dashed.
4. Degenerate button: collinear vertices $\rightarrow$ no fill, no crash.

Script: drag a vertex across another; the triangle flips; signed area changes sign; if they used unsigned area, the fill dies. Prefer signed + `abs` in the denominator, or skip if area ≈ 0 .

Lab

1. `barycentric(p, a, b, c)  $\rightarrow$  {a, b, g} OR null`.
2. `fillTriangle(img, a, b, c, colorA, colorB, colorC)`.
3. A quad = two triangles. Shared edge must not crash.
4. Optional: DDA line for wireframe overlay.

Done when the centroid is the mix of the three vertex colors.

Homework

1. Eight tests for barycentric: three vertices, centroid, clearly outside, on edge, degenerate.
 2. Written: why pixel **centers**, not integer corners.
 3. Written: relate `orient` / signed area to α .
-

Quiz (10 min)

1. Barycentric of a vertex (say a)? (2 pts)
 2. Barycentric of the centroid? (2 pts)
 3. Degenerate triangle: what does the code do? (2 pts)
 4. Why a bounding box loop is not enough without the inside test? (4 pts)
-

Common mistakes

- Testing `$\alpha + \beta + \gamma == 1$` in floats without epsilon, and rejecting everyone.
 - Using vertex pixels as integers then testing integer corners (holes).
 - `area` unsigned, then a CW triangle never fills.
 - Nested loops over the **whole canvas** for every triangle (fine for one triangle; death for a mesh — box it).
-

Board drawings

1. Triangle with α , β , γ labeled at a point.
2. Two triangles of a quad; shared edge.
3. Pixel-center grid overlay.

Week 04 Vectors and Frames

Week 4 — Vectors, points, and frames

Time: 75 min lecture + 60 min live coding

Kernel this week: `vec3` (`dot`, `cross`, `normalize`, `add`, `sub`, `scale`)

Board first: a point is not an arrow

Timing

Minutes	Do this
0–10	Quiz Week 3
10–30	Points, vectors, normals
30–50	Dot, cross, bases
50–65	Frames and change of basis
65–75	2D rotation two ways: trig vs matrix

Learning goals

1. Refuse to add two points; subtract points to get a vector.
 2. Compute dot and 3D cross; interpret 2D cross as signed area.
 3. Build a right-handed orthonormal basis from a look direction and an up hint.
 4. Say what a coordinate frame is.
 5. Rotate a 2D point with a matrix.
-

1. Types (20 min)

Object	Homogeneous w	Transforms with
Point	1	Affine matrix M
Vector / direction	0	Linear part of M (no translation)
Normal	—	<code>transpose(inverse(M))</code> on the 3×3 (Week 6/10)

Normals are not directions. Scaling an ellipse does not scale its normals the same way. Draw this; implement the normal matrix when lighting starts.

Computational geometry's `orient(a,b,c)` is `sign(cross(b-a, c-a))`. Same algebra.

2. Products (20 min)

Dot: $a \cdot b = |a| |b| \cos \theta$. Projection; Lambert will be $n \cdot l$.

Cross (3D): perpendicular to both; direction by right-hand rule. $|a \times b| = |a| |b| \sin \theta$.

2D cross (scalar): $a_x b_y - a_y b_x$.

Unit vector: $\text{normalize}(v) = v / |v|$. Zero vector: do not divide; return a policy (skip the vertex, or $(0, 1, 0)$).

3. Frames (15 min)

A frame is an origin plus three axes. Object space is a frame. World space is a frame. The model matrix **is** the object frame written in world coordinates (columns = axes and translation, for column vectors).

Change of basis: coordinates in frame B from coordinates in frame A via the matrix whose columns are A's axes in B.

Do not drown them in abstract linear algebra. One picture: a cube's local x-axis drawn in the world.

Live coding (60 min)

1. Draw a 2D origin and a point p.
 2. Rotate p with $\cos/\sin$.
 3. Rotate p with a 2×2 matrix. Identical.
 4. Draw i and j after rotation (the frame).
 5. Unit tests in the console: dot orthogonal = 0, cross of $i, j = k$.
-

Lab

Implement `vec3.js` with tests:

- `dot((1,0,0),(0,1,0)) === 0`
- `cross((1,0,0),(0,1,0)) === (0,0,1)`
- `normalize((0,0,0))` does not return NaN
- `sub` of two points is a vector; adding a point and a vector is a point (document in comments)

Visualizer: draggable 2D vector, show length and a unit arrow.

Homework

1. `vec3` tests as above plus `lerp(a, b, t)` for **points**.
 2. Written: why `p + q` for two points is meaningless (unless you secretly meant midpoint with $1/2$).
 3. Written: right-hand rule for `cross(i, j)`.
-

Quiz (10 min)

1. Point minus point is a ...? (2 pts)
 2. `dot` of two unit vectors is ...? (2 pts)
 3. 2D cross of (2,0) and (0,3)? (2 pts)
 4. Do normals transform with M? Yes/no and the correct name. (4 pts)
-

Common mistakes

- Normalizing in place and reusing the zero vector.
 - Left-handed cross by swapping arguments "until the cube looks right."
 - Treating RGB as a vector space for lighting this week (wait for Week 10).
-

Board drawings

1. Point vs vector.
2. Cross product right-hand rule.
3. Object axes drawn in the world.

Week 05 Homogeneous Transforms

Week 5 — Homogeneous coordinates and affine transforms

Time: 75 min lecture + 60 min live coding

Kernel this week: `mat4` multiply, `translate`, `rotateX/Y/Z`, `scale`

Board first: rotate-then-translate vs translate-then-rotate

Timing

Minutes	Do this
0–10	Quiz Week 4
10–25	Affine maps; why 4×4
25–50	Homogeneous points and directions
50–65	Composition order
65–75	Inverse of T, R, S

Learning goals

1. Write a translation as a 4×4 matrix.
 2. Explain `w = 1` vs `w = 0`.
 3. Compose transforms and predict the picture.
 4. Rotate a shape about its center.
 5. Invert T, R, uniform S without calling a generic inverse (optional generic inverse in the kernel).
-

1. Affine and homogeneous (15 min)

An affine transform is linear part plus translation:

$$x' = Lx + t$$

Homogeneous:

$$\begin{bmatrix} L & t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ 1 \end{bmatrix} = \begin{bmatrix} Lx + t \\ 1 \end{bmatrix}$$

Directions use `w = 0` so `t` does not apply. Lights-as-directions and view rays need this.

2. Building blocks (25 min)

Translate $T(t)$: identity, last column t (column-vector convention).

Scale $S(s_x, s_y, s_z)$: diagonal.

Rotate about a coordinate axis: 2D rotation block in the other two axes. Right-handed: positive rotateY turns from +Z toward +X? **Draw it.** Standard:

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Confirm with a point $(1, 0, 0)$ at small positive θ . If the picture disagrees with Three.js later, the convention drifted — fix the matrix, do not flip θ in five places.

3. Order (15 min)

Column vectors: **the matrix nearest the point acts first.**

```
T * R * p    // rotate around origin, then translate
R * T * p    // translate, then rotate around origin (orbits)
```

About an arbitrary point c :

$$T(c) * R * T(-c) * p$$

This is the lab.

Row-vector APIs (old D3D) reverse the product. We do not use them.

4. Inverses (10 min)

- $T(t)^{-1} = T(-t)$
- $R^{-1} = R^T$ (rotation)
- $S(s)^{-1} = S(1/s)$ if $s \neq 0$
- Non-uniform scale + rotation: inverse is not “invert each and reverse” unless you remember $(AB)^{-1} = B^{-1} A^{-1}$

Generic `inverse(M)` in the kernel is allowed; tests must include a known T^*R .

Live coding (60 min)

1. `mul(A,B)`, `mulVec(M, p)` with `w`.
2. A triangle at the origin; slider θ ; `Ry`.
3. Same triangle offset; show it swings around the origin.
4. `T(c)` `R T(-c)` about the centroid. Students clap when it works.

Print the matrix. Do not hide it in a library.

Lab

1. `mat4.js: identity, mul, translate, rotateY, scale`.
2. Rotate-about-center on the Week 3 triangle.
3. Tests: `T * T-1 = I` within epsilon; `R(90°) * (1,0,0) ≈ (0,0,-1)` or whatever **your** `Ry` does — write the expected vector in the test from the board.

Done when order A vs B is a button that changes the animation.

Homework

1. By hand: 2×2 rotation of $(1,0)$ by 90° . Then one 4×4 `T(1,0,0) * p` for $p=(0,0,0,1)$.
 2. Code: `rotateX`, `rotateZ`.
 3. Written: why directions use $w=0$.
-

Quiz (10 min)

1. Write `T(1,2,3)` (you may write "identity plus last column"). (3 pts)
 2. Does `T R` equal `R T`? (2 pts)
 3. How do you rotate about c ? (3 pts)
 4. w for a direction? (2 pts)
-

Common mistakes

- Row-major storage but column-vector formulas (document `m[col*4+row]` vs `m[row*4+col]` and **pick one**).
 - Translating after projection (Week 8) by accident.
 - Inverting by transposing a matrix that has translation.
-

Board drawings

1. $T R$ vs $R T$ on a stick figure.
2. Homogeneous column for a point and a vector.
3. Rotate about c as three arrows.

Week 06 Scene Graphs

Week 6 — Hierarchical models and scene graphs

Time: 75 min lecture + 60 min live coding

Kernel this week: scene node { `local`, `children` }, world matrix

Board first: Sun–Earth–Moon as three nested frames

Timing

Minutes	Do this
0–10	Quiz Week 5
10–25	Model matrix as object → world
25–50	Parent × local; traversal
50–65	Instancing; debug axes
65–75	Normal matrix teaser

Learning goals

1. Define the model matrix M .
 2. Compute $\text{world} = \text{parentWorld} * \text{local}$.
 3. Traverse a tree and draw each mesh with its world matrix.
 4. Draw local axes for debugging.
 5. Name why non-uniform scale breaks normals.
-

1. Model matrix (15 min)

M takes object-space vertices to world space. Its columns (column-vector convention) are the object's x-axis, y-axis, z-axis, and origin, all in world coordinates.

A “mesh at the origin” plus M is how every engine stores placement. glTF nodes are this. Three.js `Object3D.matrix` is this.

2. Hierarchy (25 min)

```
world(node) = world(parent) * local(node)
world(root) = local(root)
```

Solar system: Sun local = spin. Earth local = orbit * spin. Moon local = smaller orbit * spin. Draw Earth with `worldSun * localEarth`.

Robot arm: shoulder * elbow * wrist. This is the product they will write on the midterm.

Traversal: depth-first, pass `parentWorld` down. Do not recompute from the root for every node if you already have the parent ($O(n)$ vs $O(n \times \text{depth})$ is worth one sentence).

Cycles in the graph: forbid them.

3. Instancing and debug (15 min)

Same cube mesh, different M: instance. Do not clone vertex arrays.

Debug: at each node, draw the three axes (small RGB lines) using the world rotation, ignoring scale if scale is huge. This finds bad local matrices faster than any debugger.

4. Normal matrix (10 min)

If M has non-uniform scale, $n' = \text{normalize}((M^{-1})^T n)$ on the 3×3 . Uniform scale: `normalize(M_3x3 n)` is enough. Implement in Week 10; name it now so the midterm can ask "why."

Live coding (60 min)

Sun / Earth / Moon with three cubes (or three colored triangles). Time t . Sliders: orbit speeds.

Draw axes. Pause: "Earth's world matrix is not `T(orbit)` alone; it includes the Sun's spin if I parented badly." Fix parenting.

Lab

1. `Node { localMat4, mesh, children }.`
2. `function draw(node, parentWorld).`
3. Two-bone arm **or** turret on a tank.
4. Key `A` toggles axes.

Done when moving the parent moves the child, and rotating the child does not move the parent.

Homework

1. Written: expand `worldMoon` as a product of named matrices.
 2. Code: scene graph JSON loader (tiny: array of nodes with `parent` index) **or** hardcode the solar system cleanly.
 3. Written: instancing vs cloning vertices.
-

Quiz (10 min)

1. What does `M` do to a vertex? (2 pts)
 2. `world = ? * local` (2 pts)
 3. Draw a 3-node chain and write the product for the leaf. (4 pts)
 4. Why a normal matrix? One sentence. (2 pts)
-

Common mistakes

- `local * parent` (row-vector hangover).
 - Updating local from world every frame and drifting (Euler integration of matrices).
 - Scaling the parent and wondering why the child's offset explodes (it should).
-

Board drawings

1. Nested frames, Sun–Earth–Moon.
2. Tree with matrices on edges.
3. RGB axes on a tilted cube.

Week 07 Cameras

Week 7 — Cameras and the view transform

Time: 75 min lecture + 60 min live coding

Kernel this week: `lookAt(eye, target, up)`

Board first: camera axes u, v, w at the eye

Timing

Minutes	Do this
0–10	Quiz Week 6
10–30	Eye, target, up; camera basis
30–50	View matrix as inverse of camera world
50–65	Move world vs move camera
65–75	Optional Three.js orbit oracle (5 min max)

Learning goals

1. Build a right-handed camera basis from eye, target, up.
 2. Write V so that the camera sits at the origin looking down $-Z$.
 3. Explain V as `inverse(cameraWorld)`.
 4. Fly a camera around the Week 6 scene.
 5. Handle degenerate up $\parallel$ look.
-

1. Camera basis (20 min)

```
w = normalize(eye - target)    // +w points behind the camera if we look down -Z
// Alternative: f = normalize(target - eye), then w = -f
u = normalize(cross(up, w))    // depends on which w you chose - derive on the board
v = cross(w, u)
```

This course: camera looks down $-Z$ in view space. After V , the target should be on $-Z$.

Walk one numeric example: eye $(0,0,5)$, target $(0,0,0)$, up $(0,1,0)$. Then $w = (0,0,1)$, looking toward $-Z$ in world... wait: eye - target = $(0,0,5)$, normalize $(0,0,1)$. View space $+Z$ points toward world $+Z$, so the origin is in front if we look along world $-Z$. **Draw the camera at $+Z$ looking at origin.** Students get this wrong every year. Spend the time.

Standard `lookAt` (OpenGL-style) produces a V such that $V * eye = 0$ and the target lands on the $-Z$ axis.

2. View matrix (20 min)

Camera world matrix C has columns u, v, w, eye (if those are the camera's x,y,z axes). Then $V = C^{-1}$.

Closed form: rotation that maps camera axes to canonical, then translate by $-eye$.

$$V = \begin{bmatrix} u.x & u.y & u.z & -\text{dot}(u, eye) \\ v.x & v.y & v.z & -\text{dot}(v, eye) \\ w.x & w.y & w.z & -\text{dot}(w, eye) \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Match **your** u,v,w to this layout in live coding. Tests: $V * \text{vec4}(eye, 1) \approx 0$.

3. Duality (15 min)

Translating the camera right is the same (as a picture) as translating the world left. Students should not mix both in one frame without a story.

FPS: move eye in the u/w plane; recompute `lookAt` or update C and invert.

4. Three.js oracle (5 min)

Open orbit demo. Say: `camera.position` is eye; the engine builds V . Close it. Back to student `lookAt`.

Live coding (60 min)

Implement `lookAt`. Solar system from Week 6. Sliders: eye x,y,z . Button: reset to $(0, 2, 8)$.

Degenerate: up parallel to look $\rightarrow$ show NaN, then fix with a fallback up.

Lab

1. `lookAt` with tests (eye at $z=5$, target origin).
2. WASD or sliders.
3. Draw camera axes in **world** (debug).
4. Do not call Three.js `lookAt` in the submitted lab.

Done when moving the camera left moves the scene right, and the Sun stays the Sun.

Homework

1. Code: `lookAt` + three fixtures (including a tilted up).
 2. Written: V as inverse of C , 6–8 sentences.
 3. Written: what if up is parallel to look?
-

Quiz (10 min)

1. Name eye, target, up. (2 pts)
 2. After V , where is the eye? (2 pts)
 3. Why $-Z$ look in this course? (2 pts)
 4. $v * eye$ should be ...? (4 pts)
-

Common mistakes

- `cross(w, up)` vs `cross(up, w)` until it “looks okay” — pick the board order.
 - Leaving V as identity and moving the model instead, then calling it a camera.
 - Up vector not normalized; then v is scaled and the picture shears.
-

Board drawings

1. Camera at $+Z$, axes u, v, w .
2. V as “change to camera frame.”
3. Duality: camera right vs world left.

Week 08 Midterm and Projection

Week 8 — Midterm and perspective projection

Time: 60–75 min midterm, then 30–45 min lecture + remaining live coding

New kernel: perspective, ortho

Lab: cube that foreshortens

Hand out the **midterm topic list at the end of Week 7**. No new project work this week except keeping notes.

Midterm (60–75 min)

Written. No laptop. 100 points.

Suggested paper

Q1. Pipeline (16 pts)

Six spaces in order. Which multiply is $P * V * M$? Canvas origin vs world origin.

Q2. Color / raster (16 pts)

over formula. Barycentric of the centroid. Pixel centers.

Q3. Vectors and matrices (24 pts)

Point vs vector (w). $T R$ vs $R T$. Rotate about c . One 2×2 rotation by hand.

Q4. Scene graph (20 pts)

Three-node chain; world matrix of the leaf. What M means.

Q5. Camera (24 pts)

lookAt inputs. $v * eye$. Degenerate up. Why normals need a different matrix (name only).

Grading note

Partial credit for a correct picture with a wrong name. No credit for “the GPU does it” on a space question.

Lecture — Projection (remainder)

Orthographic

Box to clip cube. No foreshortening. UI, CAD, debug.

Perspective

Frustum: `fov` (vertical, radians), `aspect`, `near`, `far`.

Teaching matrix (OpenGL-style clip, column vectors) — write it once, test with a point on the center line:

```
sy = 1 / tan(fov/2)
sx = sy / aspect

P = [[sx, 0, 0, 0],
     [0, sy, 0, 0],
     [0, 0, a, b],
     [0, 0, -1, 0]]
```

with `a`, `b` from near/far (derive or quote Shirley / Scratchapixel). The last row `(0,0,-1,0)` copies `-z` into `w` so divide later does perspective.

Do not divide by `w` today in a vertex shader story. Divide is Week 9. Today: build `P`, multiply, **look at clip.w**.

Near > 0. Far > near. `near = 0` is forbidden.

Live coding

1. `ortho(l,r,b,t,n,f)` and `perspective(fov, aspect, n, f)`.
2. Course cube, `M = identity`, `V = lookAt`, `P = perspective`.
3. Toggle ortho.
4. Print `p_clip` for one vertex.

If the cube is behind the camera, `V` is wrong — do not “fix” `P`.

Lab

1. Implement `perspective` with a unit test: a point on the camera’s look axis has `ndc.x ≈ 0` after divide (Week 9 can finish divide; this week you may divide in JS for the test).
2. Slider fov. Cube must grow when fov shrinks.
3. Aspect from canvas width/height.

Done when ortho looks like a flat engineering drawing and perspective recedes.

Homework

1. Written: why far/near too huge hurts depth (tease Week 9).

2. Code: `ortho` + toggle.
 3. Midterm reflection optional: one mistake you will not repeat.
-

Quiz

None (midterm week). Next quiz is Week 9 on projection + last week's camera.

Common mistakes

- fov in degrees passed to `tan`.
 - aspect = height/width.
 - P with +1 in the w-row while looking down $-Z$, inconsistent with V.
-

Board drawings

1. Frustum vs ortho box.
2. Similar triangles: smaller when farther.
3. Clip `w` as "the z that perspective needs."

Week 09 Clip NDC Depth

Week 9 — Clip, NDC, viewport, depth buffer

Time: 75 min lecture + 60 min live coding

Kernel this week: perspective divide, viewport, z-buffer rasterizer

Board first: clip cube → square NDC → canvas, plus a y-flip

Timing

Minutes	Do this
0–10	Quiz: camera + projection leftovers
10–25	Clip space and near-plane
25–40	NDC and viewport (y-flip)
40–60	Depth buffer
60–75	Z-fighting; debug views

Learning goals

1. Perspective-divide clip → NDC.
 2. Map NDC to pixels with a y-flip for Canvas.
 3. Reject or skip triangles with $w \leq 0$ / behind the near plane (policy).
 4. Implement a z-buffer with a documented compare ($<$ or $\leq$).
 5. Explain z-fighting as precision vs a stretched near/far.
-

1. Clip and near (15 min)

After $p_{clip} = P V M p$, the clip volume is (typical GL) $-w \leq x, y, z \leq w$.

If a triangle straddles the near plane, a naive divide produces garbage. **Course policy:** drop the triangle if any vertex has $w \leq \epsilon$ or ndc out of range. Optional extra: lerp-clip against $z = \text{near}$. Full clipper is not required.

2. NDC and viewport (15 min)

```
ndc = clip.xyz / clip.w
```

NDC x,y,z in $[-1,1]$ if inside the volume.

Canvas:

```
sx = (ndc.x * 0.5 + 0.5) * width
sy = (1 - (ndc.y * 0.5 + 0.5)) * height // y-flip
```

Depth for the buffer: `z01 = ndc.z * 0.5 + 0.5` (0 near or 1 near — **pick one**, match compare).

3. Z-buffer (20 min)

Array `depth[width*height]`, init to $+\infty$ (or 1.0 if using $[0,1]$ far).

For each pixel in the triangle:

```
z = interpolate (barycentric of clip-space z or ndc z)
if z < depth[i]: // closer
    depth[i] = z
    putPixel(...)
```

Perspective-correct z is a later extra. Affine z in NDC is acceptable for the lab if documented.

Two overlapping triangles: the nearer color wins. Disable depth → painter's order bug.

4. Z-fighting (10 min)

Coplanar surfaces, or near too small and far too large: bits of z collapse. Fix: push near out, pull far in, add a polygon offset **name only**, or don't stack two identical planes.

Live coding (60 min)

Full cube: 12 triangles, PVM, divide, viewport, barycentric in **screen space**, z-buffer. Debug mode: `color = gray(z)`.

Show: without depth, back faces scribble; with depth, a solid cube. Culling (Week 10) is optional if depth is correct.

Lab

1. `viewport(ndc, width, height)` with y-flip tests.
2. Z-buffer cube.
3. Keys: toggle depth, toggle z-visualize.

4. A second triangle piercing the cube; occlusion must be correct.

Done when a TA rotates the cube (slider) and holes do not pop from missing depth.

Homework

1. Written: why `near = 0.0001` is a bad idea.
 2. Code: init depth; compare documented.
 3. Written: y-flip formula, one picture.
-

Quiz (10 min)

1. NDC from clip? (2 pts)
 2. Why flip y for Canvas? (2 pts)
 3. Depth init value? (2 pts)
 4. Two triangles overlap. Who wins? (4 pts)
-

Common mistakes

- Dividing before P (nonsense).
 - Using `clip.z` as a pixel depth without mapping.
 - `sy` without flip: cube is upside-down, then they negate `Ry` forever.
 - Clearing color but not depth.
-

Board drawings

1. Clip cube, divide, square.
2. NDC y-up vs canvas y-down.
3. Z-buffer as a second image.

Week 10 Lambert Lighting

Week 10 — Lighting I (Lambert)

Time: 75 min lecture + 60 min live coding

Kernel this week: Lambert, face and vertex normals, normal matrix

Board first: `n·l` on a cube face, light arrow, clamped at 0

Timing

Minutes	Do this
0–10	Quiz Week 9
10–25	Normals vs positions
25–50	Lambert, ambient, spaces
50–65	Flat vs Gouraud
65–75	Culling vs lighting

Learning goals

1. Compute a face normal from three vertices (`cross`).
 2. Shade with `max(0, n·l)` in a chosen space (world **or** view — freeze one).
 3. Apply the normal matrix when M has scale.
 4. Compare flat and Gouraud on the cube.
 5. Distinguish back-face culling from “the face is black because $n·l < 0$.”
-

1. Normals (15 min)

Face normal: `normalize(cross(b-a, c-a))` with CCW vertices.

Vertex normals: average adjacent faces, or from the mesh file. A sphere needs vertex normals; a faceted cube can use face normals.

Transform: `n_world = normalize(normalMat * n_obj)` with `normalMat = transpose(inverse(M_3×3))`.

2. Lambert (25 min)

Directional light: `l` is a unit vector **toward** the light (or from the surface — pick and write it on the board).

```
diff = kd * max(0, dot(n, l))
color = ka * ambient + diff * lightColor
```

`ka` is a policy so unlit faces are not pitch black on a projector.

Space: transform n and l into the same space. Mixing world n with view l is the classic bug.

Point lights: `l = normalize(lightPos - p)`. Need a position p at the vertex or (later) in the fragment. This week: directional is enough; point light is the lab extra.

3. Interpolation (15 min)

Flat: one n per triangle, one color, fill.

Gouraud: Lambert at vertices, barycentric-interpolate **color**. Fast; highlights smear (Week 11).

Phong interpolation: interpolate **normals**, normalize, shade per pixel. Better; do this in Week 11 if Gouraud is already working.

4. Culling (10 min)

Back-face: `dot(n_view, viewDir)` or screen-space winding. Culled faces are not drawn. A front face with `n·l < 0` is still drawn, just dark. Students conflate these.

Live coding (60 min)

Cube, face normals, one directional light, sliders for light angles. Ambient slider. Toggle culling (2D winding of the projected triangle).

Debug: `color = n * 0.5 + 0.5`.

Lab

1. Face-Lambert cube.
2. Gouraud with vertex normals (cube can reuse face normals at vertices — it will look faceted; that is OK).
3. `normalMatrix(M)`.
4. Debug normals view.

Done when rotating the light moves the terminator, and the debug-normal view is colorful, not black.

Homework

1. Written: derive why $(M^{-1})^T$ for normals (short; Shirley). Or: one picture of non-uniform scale on a circle.
 2. Code: `lambert(n, 1, kd, ka)`.
 3. Point light extra: falloff optional, not required.
-

Quiz (10 min)

1. Face normal formula. (2 pts)
 2. `max(0, n·l)` — why max? (2 pts)
 3. Same space for n and l . Why? (3 pts)
 4. Culling vs negative Lambert. (3 pts)
-

Common mistakes

- Lighting in object space while the camera moved.
 - Not normalizing n after interpolation (Week 11) or after the normal matrix.
 - `1` pointing toward the surface, then wondering why the cube is black.
-

Board drawings

1. Face with n and l , θ .
2. Non-uniform scale, wrong vs right normals.
3. Gouraud colors at vertices, blend inside.

Week 11 Blinn Phong and Gamma

Week 11 — Blinn-Phong and gamma

Time: 75 min lecture + 60 min live coding

Kernel this week: Blinn-Phong; linear lighting; gamma encode

Board first: reflection r vs half-vector h

Timing

Minutes	Do this
0–10	Quiz Week 10
10–30	Phong vs Blinn-Phong
30–45	Multiple lights; clamp vs HDR teaser
45–65	Linear vs sRGB policy for this course
65–75	Debug views (normals, lighting only)

Learning goals

1. Write Blinn-Phong with a half-vector.
 2. State that this model is not energy-conserving PBR.
 3. Sum two lights.
 4. Light in linear space; encode `pow(c, 1/2.2)` at the end.
 5. Produce a normals-as-color debug view.
-

1. Specular (20 min)

Viewer $v = \text{normalize}(\text{eye} - p)$ (world or view — same space as n).

Phong: $r = \text{reflect}(-l, n)$, $\text{spec} = (\max(0, r \cdot v))^{\text{shininess}}$.

Blinn: $h = \text{normalize}(l + v)$, $\text{spec} = (\max(0, n \cdot h))^{\text{shininess}}$.

Cheaper and the default in most real-time teaching. Shininess 16–64 for a cube; 256 is a pin spark.

```
color = ka * ambient
      + kd * max(0, n·l) * lightColor
      + ks * pow(max(0, n·h), shininess) * lightColor
```

If $n \cdot l \leq 0$, skip specular (light is behind the surface).

PBR (Semester 4): microfacets, energy conservation, IBL. Name it. Do not implement Cook-Torrance this week.

2. Many lights (15 min)

Sum contributions. uint8 will clip: `min(1, color)` or store float and encode once. HDR / bloom is Real-Time Rendering. Here: clip and mention the limitation in the report.

3. Gamma (20 min)

Policy from Week 2, now enforced:

1. Textures / vertex colors: treat 8-bit as sRGB $\rightarrow \text{pow}(c/255, 2.2)$ (approx).
2. Lights and Lambert in linear.
3. Output: `pow(linear, 1/2.2)` then $\times 255$.

Toggle in the demo: wrong (light in sRGB) vs right. The wrong one often “looks contrasty” and students prefer it — tell them the projector lies, the policy stands.

Full sRGB curve (2.4 piecewise) is optional extra credit, not required.

Live coding (60 min)

Blinn-Phong cube, shininess slider, two lights, gamma toggle. Debug: specular-only ($k_d=0$).

Per-pixel Phong interpolation if Gouraud from last week is done: interpolate n , normalize, shade.

Lab

1. `blinnPhong(...)` with tests: $n=l=v \rightarrow \text{spec} > 0$; n opposite $l \rightarrow \text{spec} = 0$.
2. Two directional lights.
3. Gamma encode function.
4. Debug keys: N (normals), L (diffuse only), S (spec only).

Done when the highlight **moves with the camera** (depends on v). If it does not, they used a constant v .

Homework

1. Written: h vs r , one picture.

2. Written: why lighting in sRGB is wrong (one paragraph).
 3. Code: shininess slider documented in README.
-

Quiz (10 min)

1. Half-vector formula. (2 pts)
 2. Why skip spec when $n \cdot l < 0$? (2 pts)
 3. Where does gamma encode happen? (3 pts)
 4. Is Blinn-Phong PBR? Yes/no. (3 pts)
-

Common mistakes

- v as $(0,0,1)$ forever.
 - Adding spec in sRGB then gamma-encoding again.
 - Shininess 0.5 because they thought it was a 0–1 roughness (it is an exponent).
-

Board drawings

1. n , l , v , r , h .
2. Linear add vs sRGB add.
3. Two lights, two highlights.

Week 12 Texture Mapping

Week 12 — Texture mapping

Time: 75 min lecture + 60 min live coding

Kernel this week: UV, `sampleNearest`, textured Lambert

Board first: a quad with (0,0),(1,0),(1,1),(0,1) labeled

Timing

Minutes	Do this
0–10	Quiz Week 11
10–25	UV, wrap, clamp
25–45	Sampling: nearest, bilinear (idea)
45–60	Perspective-correct interpolation (picture)
60–75	Texture as albedo; minification / mipmaps named

Learning goals

1. Attach UV to vertices and interpolate with barycentric.
 2. Sample a texture with nearest neighbor and a wrap/clamp policy.
 3. Multiply albedo by Lambert (texture is not lighting).
 4. Explain why affine UV looks wrong in perspective.
 5. Name bilinear and mipmaps without implementing mipmaps.
-

1. UV (15 min)

UV in $[0,1]$ typically. $(0,0)$ is which corner of the **image**? Canvas/PNG: often top-left. glTF: often bottom-left. **Pick one, write it on the board, stick to it.** If the texture is upside-down, flip v: $v = 1 - v$, do not randomly swap UVs on four vertices.

Wrap: $u = u - \text{floor}(u)$ (repeat). Clamp: $u = \min(1, \max(0, u))$.

2. Sampling (20 min)

Nearest:


```
x = floor(u * width)
y = floor(v * height)
```

with wrap/clamp before that.

Bilinear: four taps, lerp. Name; optional lab extra.

Minification: many texels per pixel → aliasing. Mipmaps: prefiltered levels. Required knowledge: the **word** and why games use them. Not required code.

3. Perspective-correct (15 min)

Affine barycentric in **screen space** interpolates UV as if the triangle were flat on the screen. A perspective quad (road, floor) shows the classic bow-tie UV.

Teaching fix: interpolate u/z , v/z , $1/z$, then $u = (u/z) / (1/z)$. z from clip or view. Extra credit if they implement it; midterm-level: **describe** it.

4. Shading (10 min)

```
albedo = sample(tex, uv) // linear if following Week 11
color = lambert * albedo + specular * light // spec often not multiplied by albedo (policy)
```

A texture is not a reason to skip normals.

Live coding (60 min)

Load a PNG into `ImageData` (or a procedural checker if `file://` blocks). Textured quad facing the camera. Then UVs on the cube (each face 0–1). Mag filter toggle nearest / bilinear if time.

UV debug: `putPixel RGB = (u, v, 0)`.

Lab

1. `sampleNearest(image, u, v, mode)`.
2. Textured cube **or** a floor quad in perspective (to see affine bug).
3. UV debug view.
4. Procedural checker fallback.

Done when rotating the cube does not smear a single texel across a face unless UVs are actually constant (a bug).

Homework

1. Written: affine vs perspective-correct UV, one picture of a trapezoid.
 2. Code: clamp vs repeat demo.
 3. Written: mipmaps in three sentences.
-

Quiz (10 min)

1. UV of a vertex — is it a position? (2 pts)
 2. Nearest sample formula. (2 pts)
 3. Why affine UV fails in perspective. (4 pts)
 4. Texture replaces lighting? Yes/no. (2 pts)
-

Common mistakes

- Integer UV 0–width mixed with 0–1.
 - Sampling with v from the wrong origin; “fixing” by rotating the PNG.
 - Perspective cube with one UV per cube (all faces the same smear).
-

Board drawings

1. UV square on a mesh.
2. Nearest vs bilinear 2×2 .
3. Trapezoid road, affine vs correct.

Week 13 GPU Mapping

Week 13 — The same scene on the GPU

Time: 75 min lecture + 60 min live coding

Live coding: WebGL2 cube = student PVM + Lambert + texture

Lab: project checkpoint (vertical slice)

Homework: project only

Timing

Minutes	Do this
0–10	Quiz Week 12
10–30	CPU function → GPU stage table
30–50	WebGL2: attributes, uniforms, depth, cull
50–65	Where Three.js hides V and P
65–75	Picking is geometry, not “the engine”

Learning goals

1. Map every stage of the software renderer to a GPU name.
 2. Write (or complete) a WebGL2 program for the course cube.
 3. Point to `u_proj * u_view * u_model` in a shader.
 4. Use Three.js only as a labeled oracle, not as the checkpoint.
 5. Freeze project scope.
-

1. The payoff table (20 min)

Walk slowly. One sentence from this course, one from WebGL.

Student CPU	GPU / API
<code>putPixel</code>	fragment writes to framebuffer
barycentric fill	rasterizer (fixed)
z-buffer	<code>DEPTH_TEST</code>
<code>P * V * M</code>	vertex shader <code>gl_Position</code>
Lambert / Blinn	fragment shader
<code>sampleNearest</code>	<code>texture()</code> + sampler
scene graph	CPU updates uniforms / instancing
y-flip viewport	viewport + NDC; canvas still y-down in UI

Open 01 Conventions and index.html. Demos 01–04 and 06 are the path. Students may copy boilerplate; they must **fill** `u_model`, `u_view`, `u_proj` from their `mat4` code if they claim the math.

2. WebGL2 minimum (20 min)

- Compile / link; print shader logs
- One `vec3` position attribute, one `vec2` uv
- Uniforms: three `mat4`, light dir, sampler
- `enable(DEPTH_TEST)`, `enable(CULL_FACE)`, CCW
- Black-screen checklist from conventions (canvas size, camera, near plane, winding)

Vertex shader is Weeks 5–9. Fragment shader is Weeks 10–12.

3. Three.js (15 min)

`camera.projectionMatrix` is P. `camera.matrixWorldInverse` is V. `mesh.matrixWorld` is M. `MeshPhongMaterial` is Week 11. `Raycaster` is computational geometry, not a lighting topic.

If they submit a Three.js scene for the **checkpoint**, they still need a one-page table mapping Object3D to their software renderer. No table, incomplete.

4. Picking (10 min)

Click → ray → triangle (CG Weeks 2, 9, 13). The GPU picture does not replace the predicate.

Live coding (60 min)

Port the course cube to WebGL2. Same texture as Week 12. Move the camera with the same `lookAt`. Side-by-side if possible: canvas software vs WebGL.

When it mismatches: NDC y, winding, linear/sRGB, light space.

Lab (checkpoint)

Must be true:

- Mesh (at least the cube)
- Camera (`lookAt` or documented Three.js camera)
- One light
- One texture **or** a debug UV/normal view
- Depth correct
- README: how to run; which files are student math vs library

Software-only slice is **complete** if WebGL failed. WebGL-only slice is complete if they can explain each uniform. Both is the A path.

Homework

Project. Written: CPU → GPU table, one page, in the repo.

Quiz (10 min)

1. Where does `gl_Position` sit in the space chain? (2 pts)
 2. Depth test replaces which CPU structure? (2 pts)
 3. Name one black-screen cause. (2 pts)
 4. `matrixWorldInverse` is which matrix? (2 pts)
 5. May Three.js Raycaster be the student picking algorithm? (2 pts)
-

Common mistakes

- Using Three.js for the checkpoint with no mapping table.
 - sRGB texture sampled as linear in WebGL (gamma surprise vs CPU).
 - Growing EPS / fov until the cube "shows up" instead of fixing V.
-

Board drawings

1. Payoff table (leave the GPU column empty; fill with the class).
2. Vertex vs fragment shader boxes.
3. Same cube, two windows: CPU / GPU.

Week 14 Project Studio

Week 14 — Project studio

Time: 30 min lecture, then studio until the period ends

No new theory

Required this week: running pipeline, one debug view, README

Timing

Minutes	Do this
0–15	How to write the report
15–30	Defense-style questions; presentation clock
30–end	Desk review. Professor and TAs circulate.

Do not give a lecture on PBR. If a team is stuck on a matrix, debug the matrix.

Learning goals (for the studio)

1. Freeze scope: one pipeline story, one visual.
 2. Write a report a second examiner can grade without watching the demo twice.
 3. Answer “what is in clip space?” without guessing.
-

1. Report structure (15 min)

6–8 pages, including figures. Not a Three.js tutorial.

Section	What we want	Typical length
Problem	What picture are you making?	½ page
Related course ideas	Which weeks; what you did not use	½ page
Pipeline	Spaces, matrices, raster or shaders	2 pages
A hard case	Near plane, winding, gamma, affine UV	½–1 page
Implementation	Math kernel vs library	1 page
Results	Screenshots, debug views	1 page
Limitations	Honest	½ page
References	Shirley, Scratchapixel, WebGL spec, docs	½ page

Figures must have captions. "Figure 2. NDC y-flip."

Do not paste 200 lines of shader. A 15-line vertex shader is enough.

Do not invent timings. If you did not measure FPS, say so.

2. Questions I will ask next week (15 min)

1. Point to M, V, P in your code. What is the product order?
2. Where do you divide by w?
3. Show a debug view (normals / UV / depth). What would a black normal view mean?
4. If I disable depth, what happens?
5. What did a library do, and what did you write?
6. Affine vs perspective-correct UV: did you implement it? If not, where would it show?
7. Linear vs sRGB: where do you encode?
8. What would break in a left-handed API?

Presentation clock (Week 15)

- 12 minutes demo + story
- 5 minutes questions
- Hard stop. Rehearse once this week with a TA timer.

Min	Content
0–2	Problem and one picture
2–6	Pipeline, one matrix identity, one limitation
6–10	Live demo, including a debug view
10–12	Who did what; what you would do next

3. Studio rules

Must be true before they leave today

- README runs on a lab machine (`npx serve / python -m http.server`).
- The scene is more than a single untransformed triangle **or** they have a written waiver from you for a documented recovery plan.
- One debug view works.
- README: who implemented which file.

Desk review order

1. `mat4` / shaders

2. Camera
3. Rasterizer or depth test
4. Report outline

Comment on the **pipeline**, not the CSS.

Scope cuts

If they are behind	Cut
glTF / PBR / shadows	Cube + Lambert + texture
Dual CPU/GPU	One path, mapping table still required
Scene graph city	Two nodes
Beautiful post	Unlit textured cube with correct PVM

Lab

The studio **is** the lab. Checkpoint: complete / incomplete from Week 13 plus today's README/debug-view check.

Homework

Finish the report draft and the 30-second recording. Submit the recording before Week 15.

Quiz

None.

Common failure modes

- Pretty Three.js, no student matrix.
- Report is a Blender tutorial.
- No debug view.
- Three people, one git author.

Board

Write only:

1. The 8 report headings

2. The 8 defense questions

3. The 12 + 5 clock

Week 15 Presentations

Week 15 — Project presentations

Time: full session of 12 + 5 minute slots

No new lecture

Deliverables: live demo, repo, 6–8 page report, 30-second recording

Before the session

- Room: projector + spare dongle.
 - Load each team's recording as backup.
 - Print the rubric one sheet per team.
 - Order: software rasterizers first, then WebGL ports, so comparison is fair.
 - A TA keeps time: 10-minute warning, 12-minute stop.
-

Slot format

Time	Who	What
12 min	team	problem, pipeline, live demo, limitations
5 min	examiner	two questions from the Week 14 list, plus one specific to their code
1 min	—	switch

If the demo dies, play the recording. Do not debug for five minutes in front of the class.

Required deliverables

1. **Live demo** of the student pipeline (software and/or WebGL they can explain).
2. **Repository** with README, math kernel or shaders, debug view, library note.
3. **Report** 6–8 pages, captions on figures.
4. **30-second recording** for the IGWT portfolio.

Late report: department rule from Week 1. Suggested: –10% per 24 hours, zero after 72 hours.

Rubric (copy to a sheet)

Criterion	Weight	0–5 notes
Correct pipeline (tests + debug view)	30%	
Visual explanation of a stage	20%	
Code quality and repository	15%	
Report	20%	
Presentation and live demo	15%	

After the last talk

Say:

Computer graphics in this program is not a catalog of engine checkboxes.

It is the habit of:

1. naming the space,
2. writing the matrix product,
3. looking at a debug view,
4. only then switching on a library.

That habit is what they take into WebGL, Three.js, Real-Time Rendering, and the capstone.

Quiz / homework

None. Grades close when the report and repo are in.

Board

The rubric table. The clock. Nothing else.

Extra exercises — index

Computer Graphics I — extra exercises and snippets

Parent: 00 Lectures

Open demos at [code/index.html]. Copy from [code/kernel.js]. These sheets are extra (recitation, makeup, exam bank). Each week note already has lab / homework / quiz.

Week	Sheet	Demo
1	Week 01	[01-putpixel]
2	Week 02	[02] [03]
3	Week 03	[04] [05]
4	Week 04	[06-vec3]
5	Week 05	[07] [08]
6	Week 06	[09-scene-graph]
7	Week 07	[10-lookat]
8	Week 08	[11-perspective]
9	Week 09	[12-zbuffer]
10	Week 10	[13-lambert]
11	Week 11	[14-blinn-phong]
12	Week 12	[15-texture]
13	Week 13	[16-webgl-cube]
14	Week 14	[18-project-sandbox]
15	Week 15	[17-kernel-tests]

Recitation: 3 written problems + 1 snippet. Hidden fixtures: [17-kernel-tests.html].

Extra exercises — Week 01

Extra exercises — Week 1 (images / pipeline)

Lecture: Week 01 What Computer Graphics Is · Demo: [01-putpixel]

Written

1. Six spaces in order. One sentence each.
2. Rasterization vs ray tracing: who loops over triangles first?
3. `P * V * M` or `M * V * P` in this course? Why?
4. Canvas `(0,0)` vs world origin.
5. Why `canvas.width` must match the backing store, not only CSS size.
6. Index of pixel (3, 2) in a width-10 RGBA buffer.
7. What we will not implement this term (name three).

Coding

8. `putPixel` with bounds checks. Tests: corner, off-canvas, 1×1 .
9. `clear` then a clipped rectangle. Do not wrap with %.

```
export function putPixel(img, x, y, r, g, b, a = 255) {
  x = x | 0; y = y | 0;
  if (x < 0 || y < 0 || x >= img.width || y >= img.height) return;
  const i = (y * img.width + x) * 4;
  img.data[i] = r; img.data[i+1] = g; img.data[i+2] = b; img.data[i+3] = a;
}
```

Extra exercises — Week 02

Extra exercises — Week 2 (color)

Lecture: Week 02 Color and Framebuffer · Demos: [02] [03]

Written

1. `over` for rgb, alpha in $[0,1]$. Ignore output alpha.
2. Is byte 128 half as much light as 255?
3. CSS-stretched canvas: why a circle becomes an ellipse.
4. Straight vs premultiplied: one sentence.
5. Letterbox vs stretch a 16:9 buffer in a 4:3 window.

Coding

6. `overPixel` tests: opaque, alpha 0, 50% red on black.
7. Eight boxes with alpha $1/8$... $8/8$.

```
out_rgb = src_rgb * src_a + dst_rgb * (1 - src_a)
out_a   = src_a + dst_a * (1 - src_a)
```

Extra exercises — Week 03

Extra exercises — Week 3 (triangles)

Lecture: Week 03 Lines and Triangles · Demos: [04] [05]

Written

1. Barycentric of a vertex; of the centroid.
2. Why pixel **centers** ($x+0.5$, $y+0.5$) ?
3. Degenerate triangle: what does the code do?
4. Relate signed area / `orient` to α .
5. Why a bounding box loop is not enough.

Coding

6. `barycentric` tests: three vertices, centroid, outside, on edge, collinear.
7. Quad as two triangles; shared edge must not crash.

```
function barycentric(p, a, b, c) {  
  const area = (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);  
  if (Math.abs(area) < 1e-8) return null;  
  const alpha = ((b.x - p.x) * (c.y - p.y) - (b.y - p.y) * (c.x - p.x)) / area;  
  const beta = ((c.x - p.x) * (a.y - p.y) - (c.y - p.y) * (a.x - p.x)) / area;  
  return { a: alpha, b: beta, g: 1 - alpha - beta };  
}
```


Extra exercises — Week 04

Extra exercises — Week 4 (vectors)

Lecture: Week 04 Vectors and Frames · Demo: [06-vec3]

Written

1. Point minus point is a ...? Point plus point?
2. `dot` of two unit vectors.
3. 2D cross of (2,0) and (0,3).
4. Do normals transform with M? Correct name.
5. Right-hand rule for `cross(i, j)`.

Coding

6. `dot` / `cross` / `normalize(0)` must not return NaN.
7. Rotate (1,0) with a 2×2 matrix; match `cos/sin`.

```
function vcross(a, b) {  
  return {  
    x: a.y * b.z - a.z * b.y,  
    y: a.z * b.x - a.x * b.z,  
    z: a.x * b.y - a.y * b.x,  
  };  
}
```

Extra exercises — Week 05

Extra exercises — Week 5 (matrices)

Lecture: Week 05 Homogeneous Transforms · Demos: [07] [08]

Written

1. Write $T(1,2,3)$ (last column).
2. Does $T \cdot R$ equal $R \cdot T$?
3. Rotate about c: three-matrix product.
4. w for a point vs a direction.
5. Inverse of T, of R, of uniform S.

Coding

6. $T \cdot T^{-1} \approx I$. $R_y(90^\circ) \cdot (1,0,0) \approx (0,0,-1)$ under this course's R_y .
7. Button: TR vs RT . The pictures must differ.

```
// column vectors: matrix nearest the point acts first
p2 = T * R * p      // rotate about origin, then translate
p2 = T(c) * R * T(-c) * p
```

Extra exercises — Week 06

Extra exercises — Week 6 (scene graph)

Lecture: Week 06 Scene Graphs · Demo: [09-scene-graph]

Written

1. What does M do to a vertex?
2. `world = ? * local`
3. Expand `worldMoon` as a product.
4. Instancing vs cloning vertices.
5. Why a normal matrix if parent has non-uniform scale.

Coding

6. Two-bone arm or turret. Moving parent moves child; not vice versa.
7. Draw RGB axes at each node.

```
function draw(node, parentWorld) {  
  const world = m4mul(parentWorld, node.local);  
  drawMesh(node.mesh, world);  
  node.children.forEach((ch) => draw(ch, world));  
}
```

Extra exercises — Week 07

Extra exercises — Week 7 (camera)

Lecture: Week 07 Cameras · Demo: [10-lookat]

Written

1. Name eye, target, up.
2. After V, where is the eye?
3. Why this course looks down -Z.
4. $v \cdot \text{eye}$ should be ...?
5. Degenerate: up $\parallel$ look. What do you do?

Coding

6. `lookAt` fixture: eye (0,0,5), target origin, $v \cdot \text{eye} \approx 0$.
7. Sliders for eye. Do not call Three.js `lookAt` in the lab.

```
w = normalize(eye - target)
u = normalize(cross(up, w)) // fallback if parallel
v = cross(w, u)
```

Extra exercises — Week 08

Extra exercises — Week 8 (projection)

Lecture: Week 08 Midterm and Projection · Demo: [11-perspective]

Makeup drills (no laptop) plus projection.

Written

1. Six spaces; barycentric of centroid; T R vs R T (midterm leftovers).
2. fov in radians. What happens if you pass degrees to `tan`?
3. aspect = width/height or the reverse?
4. Why `near = 0` is forbidden.
5. Smaller fov → larger or smaller cube on screen?

Coding

6. Toggle ortho / perspective.
7. Slider fov; cube grows when fov shrinks.

```
sy = 1 / tan(fovY / 2)
sx = sy / aspect
// last row of P copies -z into w (OpenGL-style)
```

Extra exercises — Week 09

Extra exercises — Week 9 (NDC / depth)

Lecture: Week 09 Clip NDC Depth · Demo: [12-zbuffer]

Written

1. NDC from clip?
2. Why flip y for Canvas?
3. Depth init value in this kernel (1 = far).
4. Two overlapping triangles: who wins?
5. Why `near = 0.0001` causes z-fighting.

Coding

6. Viewport tests: NDC $(-1, 1) \rightarrow$ top-left pixel.
7. Toggle depth; without it, painter order is wrong.

```
ndc = clip.xyz / clip.w
sx = (ndc.x * 0.5 + 0.5) * width
sy = (1 - (ndc.y * 0.5 + 0.5)) * height
```

Extra exercises — Week 10

Extra exercises — Week 10 (Lambert)

Lecture: Week 10 Lambert Lighting · Demo: [13-lambert]

Written

1. Face normal from three CCW vertices.
2. Why $\max(0, \mathbf{n} \cdot \mathbf{l})$?
3. Same space for $\mathbf{n}$ and $\mathbf{l}$. Why?
4. Culling vs negative Lambert.
5. Why $(\mathbf{M}^{-1})^T$ for normals.

Coding

6. Face-Lambert cube; light yaw slider.
7. Debug view $\mathbf{n} * 0.5 + 0.5$. Must not be black.

```
diff = kd * max(0, dot(n, l))  
color = ka * ambient + diff * lightColor
```

Extra exercises — Week 11

Extra exercises — Week 11 (Blinn-Phong)

Lecture: Week 11 Blinn Phong and Gamma · Demo: [14-blinn-phong]

Written

1. Half-vector formula.
2. Why skip spec when $n \cdot l < 0$?
3. Where does gamma encode happen?
4. Is Blinn-Phong PBR?
5. Why the highlight must move when the camera moves.

Coding

6. `blinnPhong` tests: facing vs back.
7. Gamma toggle. Lighting stays linear.

```
h = normalize(l + v)
spec = ks * pow(max(0, dot(n, h)), shininess)
out = pow(clamp(linear, 0, 1), 1/2.2)
```


Extra exercises — Week 12

Extra exercises — Week 12 (textures)

Lecture: Week 12 Texture Mapping · Demo: [15-texture]

Written

1. Is UV a position in world space?
2. Nearest sample formula.
3. Why affine UV fails in perspective (trapezoid picture).
4. Texture replaces lighting? Yes/no.
5. Mipmaps in three sentences.

Coding

6. `sampleNearest` clamp vs repeat.
7. UV debug coloring. Checker × Lambert.

```
x = floor(u * width)
y = floor(v * height)
albedo = sample(tex, uv)
color = lambert * albedo
```

Extra exercises — Week 13

Extra exercises — Week 13 (GPU map)

Lecture: Week 13 GPU Mapping · Demo: [16-webgl-cube]

Written

1. `gl_Position` is which space?
2. Depth test replaces which CPU structure?
3. One black-screen cause from the conventions list.
4. `matrixWorldInverse` is which matrix?
5. May Three.js `Raycaster` be the student picking algorithm?

Coding

6. Fill `u_p`, `u_v`, `u_m` from the **same** `mat4` kernel as the CPU cube.
7. One-page table: CPU function → GPU stage, in the repo.

```
gl_Position = u_p * u_v * u_m * vec4(a_pos, 1.0);
```

Extra exercises — Week 14

Extra exercises — Week 14 (studio)

Lecture: Week 14 Project Studio · Demo: [18-project-sandbox]

Desk-review drills (write in the repo)

1. Point to M, V, P. Product order?
2. Where do you divide by w?
3. Show normals / UV / depth debug. Black normals would mean ...?
4. Disable depth: what happens?
5. Library vs student files.
6. Affine UV: implemented? If not, where would it show?
7. Linear vs sRGB: where do you encode?
8. Freeze the sandbox reset as the ugly/debug scene.

Snippet — viewport y-flip fixture

```
const vp = viewport({ x: -1, y: 1, z: 0 }, 100, 50);  
assert(Math.abs(vp.x) < 1e-6 && Math.abs(vp.y) < 1e-6); // top-left
```

Extra exercises — Week 15

Extra exercises — Week 15 (presentations)

Lecture: Week 15 Presentations · Tests: [17-kernel-tests]

Rehearse 12 + 5 once. Each teammate answers one:

1. M, V, P and the product.
2. Debug view: what it proves.
3. Library vs student math.
4. One limitation (gamma, affine UV, near plane, winding).

Do not add algorithms this week. Extend [17-kernel-tests.html] if a fixture is still missing.

Code catalog

Software-rasterizer kernel and demos

Teaching Canvas `ImageData` demos for 00 Lectures.

- Open [index.html].
- Copy functions from [kernel.js]. Prefer copying into a student starter.
- [raster.js] is an rAF + `putImageData` loop, not a production engine.

Serve this folder if `file://` is blocked: `npx serve` or `python -m http.server`.

Catalog: 11 Computer Graphics Snippets

Exercises: 00 Index

kernel.js

```
/**
 * IGWT Computer Graphics I kernel (software renderer).
 * Teaching code: readable, not a production engine.
 * Copy a function into a student starter rather than importing this forever.
 *
 * Conventions: right-handed, Y-up, camera looks -Z,
 * column-major mat4, p_clip = P * V * M * p, CCW front.
 */
(function (global) {
  "use strict";

  const EPS = 1e-8;
  const CG = { EPS };

  function clamp(x, a, b) {
    return Math.max(a, Math.min(b, x));
  }

  function vec3(x, y, z) {
    return { x: x, y: y, z: z };
  }

  function vadd(a, b) {
    return { x: a.x + b.x, y: a.y + b.y, z: a.z + b.z };
  }

  function vsub(a, b) {
    return { x: a.x - b.x, y: a.y - b.y, z: a.z - b.z };
  }

  function vscale(a, s) {
    return { x: a.x * s, y: a.y * s, z: a.z * s };
  }

  function vdot(a, b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
  }

  function vcross(a, b) {
    return {
      x: a.y * b.z - a.z * b.y,
      y: a.z * b.x - a.x * b.z,
      z: a.x * b.y - a.y * b.x,
    };
  }

  function vlen(a) {
    return Math.hypot(a.x, a.y, a.z);
  }

  function vnormalize(a) {
    const L = vlen(a);
    if (L < EPS) return { x: 0, y: 1, z: 0 };
    return vscale(a, 1 / L);
  }

  function vlerp(a, b, t) {
    return vadd(vscale(a, 1 - t), vscale(b, t));
  }

  function cross2(ax, ay, bx, by) {
    return ax * by - ay * bx;
  }

  /** Column-major 4x4. */
  function m4ident() {
    return [1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1];
  }
})(global);
```

```

}
function m4mul(a, b) {
  const o = new Array(16);
  for (let c = 0; c < 4; c++) {
    for (let r = 0; r < 4; r++) {
      o[c * 4 + r] =
        a[0 * 4 + r] * b[c * 4 + 0] +
        a[1 * 4 + r] * b[c * 4 + 1] +
        a[2 * 4 + r] * b[c * 4 + 2] +
        a[3 * 4 + r] * b[c * 4 + 3];
    }
  }
  return o;
}

function m4mulVec(m, x, y, z, w) {
  w = w == null ? 1 : w;
  return {
    x: m[0] * x + m[4] * y + m[8] * z + m[12] * w,
    y: m[1] * x + m[5] * y + m[9] * z + m[13] * w,
    z: m[2] * x + m[6] * y + m[10] * z + m[14] * w,
    w: m[3] * x + m[7] * y + m[11] * z + m[15] * w,
  };
}

function m4translate(tx, ty, tz) {
  const m = m4ident();
  m[12] = tx;
  m[13] = ty;
  m[14] = tz;
  return m;
}

function m4scale(sx, sy, sz) {
  const m = m4ident();
  m[0] = sx;
  m[5] = sy;
  m[10] = sz;
  return m;
}

function m4rotateX(t) {
  const c = Math.cos(t), s = Math.sin(t);
  const m = m4ident();
  m[5] = c;
  m[6] = s;
  m[9] = -s;
  m[10] = c;
  return m;
}

function m4rotateY(t) {
  const c = Math.cos(t), s = Math.sin(t);
  const m = m4ident();
  m[0] = c;
  m[2] = -s;
  m[8] = s;
  m[10] = c;
  return m;
}

function m4rotateZ(t) {
  const c = Math.cos(t), s = Math.sin(t);
  const m = m4ident();
  m[0] = c;
  m[1] = s;
  m[4] = -s;
  m[5] = c;
  return m;
}

```

```

}
function m4lookAt(eye, target, up) {
  let w = vnormalize(vsub(eye, target));
  let u = vcross(up, w);
  if (vlen(u) < 1e-6) {
    const alt = Math.abs(up.y) < 0.9 ? vec3(0, 1, 0) : vec3(1, 0, 0);
    u = vcross(alt, w);
  }
  u = vnormalize(u);
  const v = vcross(w, u);
  return [
    u.x, v.x, w.x, 0,
    u.y, v.y, w.y, 0,
    u.z, v.z, w.z, 0,
    -vdot(u, eye), -vdot(v, eye), -vdot(w, eye), 1,
  ];
}

function m4perspective(fovY, aspect, near, far) {
  const f = 1 / Math.tan(fovY / 2);
  const nf = 1 / (near - far);
  const m = new Array(16).fill(0);
  m[0] = f / aspect;
  m[5] = f;
  m[10] = (far + near) * nf;
  m[11] = -1;
  m[14] = (2 * far * near) * nf;
  return m;
}

function m4ortho(l, r, b, t, n, f) {
  const m = m4ident();
  m[0] = 2 / (r - l);
  m[5] = 2 / (t - b);
  m[10] = -2 / (f - n);
  m[12] = -(r + l) / (r - l);
  m[13] = -(t + b) / (t - b);
  m[14] = -(f + n) / (f - n);
  return m;
}

function m4normal(m) {
  const a = m[0], b = m[1], c = m[2];
  const d = m[4], e = m[5], f = m[6];
  const g = m[8], h = m[9], i = m[10];
  const A = e * i - f * h;
  const B = f * g - d * i;
  const C = d * h - e * g;
  const det = a * A + b * B + c * C;
  if (Math.abs(det) < EPS) return m4ident();
  const inv = 1 / det;
  return [
    A * inv, (c * h - b * i) * inv, (b * f - c * e) * inv, 0,
    B * inv, (a * i - c * g) * inv, (c * d - a * f) * inv, 0,
    C * inv, (b * g - a * h) * inv, (a * e - b * d) * inv, 0,
    0, 0, 0, 1,
  ];
}

function putPixel(img, x, y, r, g, b, a) {
  x = x | 0;
  y = y | 0;
  if (x < 0 || y < 0 || x >= img.width || y >= img.height) return;
  const i = (y * img.width + x) * 4;
  img.data[i] = r;
  img.data[i + 1] = g;

```



```

    img.data[i + 2] = b;
    img.data[i + 3] = a == null ? 255 : a;
}
function clear(img, r, g, b, a) {
    r = r == null ? 28 : r;
    g = g == null ? 28 : g;
    b = b == null ? 32 : b;
    a = a == null ? 255 : a;
    const d = img.data;
    for (let i = 0; i < d.length; i += 4) {
        d[i] = r;
        d[i + 1] = g;
        d[i + 2] = b;
        d[i + 3] = a;
    }
}
function over(sr, sg, sb, sa, dr, dg, db, da) {
    const a = sa / 255, da01 = da / 255;
    const oa = a + da01 * (1 - a);
    if (oa < EPS) return [0, 0, 0, 0];
    return [
        (sr * a + dr * da01 * (1 - a)) / oa,
        (sg * a + dg * da01 * (1 - a)) / oa,
        (sb * a + db * da01 * (1 - a)) / oa,
        oa * 255,
    ];
}
function overPixel(img, x, y, r, g, b, a) {
    x = x | 0;
    y = y | 0;
    if (x < 0 || y < 0 || x >= img.width || y >= img.height) return;
    const i = (y * img.width + x) * 4;
    const o = over(r, g, b, a, img.data[i], img.data[i + 1], img.data[i + 2], img.data[i + 3]);
    img.data[i] = o[0];
    img.data[i + 1] = o[1];
    img.data[i + 2] = o[2];
    img.data[i + 3] = o[3];
}

function barycentric(p, a, b, c) {
    const area = cross2(b.x - a.x, b.y - a.y, c.x - a.x, c.y - a.y);
    if (Math.abs(area) < EPS) return null;
    const alpha = cross2(b.x - p.x, b.y - p.y, c.x - p.x, c.y - p.y) / area;
    const beta = cross2(c.x - p.x, c.y - p.y, a.x - p.x, a.y - p.y) / area;
    const gamma = 1 - alpha - beta;
    return { a: alpha, b: beta, g: gamma };
}

function fillTriangle(img, a, b, c, ca, cb, cc) {
    const minX = Math.max(0, Math.floor(Math.min(a.x, b.x, c.x)));
    const maxX = Math.min(img.width - 1, Math.ceil(Math.max(a.x, b.x, c.x)));
    const minY = Math.max(0, Math.floor(Math.min(a.y, b.y, c.y)));
    const maxY = Math.min(img.height - 1, Math.ceil(Math.max(a.y, b.y, c.y)));
    for (let y = minY; y <= maxY; y++) {
        for (let x = minX; x <= maxX; x++) {
            const w = barycentric({ x: x + 0.5, y: y + 0.5 }, a, b, c);
            if (!w || w.a < -1e-5 || w.b < -1e-5 || w.g < -1e-5) continue;
            const r = w.a * ca[0] + w.b * cb[0] + w.g * cc[0];
            const g = w.a * ca[1] + w.b * cb[1] + w.g * cc[1];
            const bl = w.a * ca[2] + w.b * cb[2] + w.g * cc[2];
            putPixel(img, x, y, r, g, bl, 255);
        }
    }
}

```

```

function viewport(ndc, width, height) {
  return {
    x: (ndc.x * 0.5 + 0.5) * width,
    y: (1 - (ndc.y * 0.5 + 0.5)) * height,
    z: ndc.z * 0.5 + 0.5,
  };
}

function toNdc(clip) {
  if (Math.abs(clip.w) < EPS) return null;
  return { x: clip.x / clip.w, y: clip.y / clip.w, z: clip.z / clip.w };
}

function makeDepth(w, h) {
  return new Float32Array(w * h).fill(1);
}

function fillTriangleDepth(img, depth, a, b, c, ca, cb, cc) {
  const minX = Math.max(0, Math.floor(Math.min(a.x, b.x, c.x)));
  const maxX = Math.min(img.width - 1, Math.ceil(Math.max(a.x, b.x, c.x)));
  const minY = Math.max(0, Math.floor(Math.min(a.y, b.y, c.y)));
  const maxY = Math.min(img.height - 1, Math.ceil(Math.max(a.y, b.y, c.y)));
  const area = cross2(b.x - a.x, b.y - a.y, c.x - a.x, c.y - a.y);
  if (area <= EPS) return;
  for (let y = minY; y <= maxY; y++) {
    for (let x = minX; x <= maxX; x++) {
      const w = barycentric({ x: x + 0.5, y: y + 0.5 }, a, b, c);
      if (!w || w.a < -1e-5 || w.b < -1e-5 || w.g < -1e-5) continue;
      const z = w.a * a.z + w.b * b.z + w.g * c.z;
      const di = y * img.width + x;
      if (z >= depth[di]) continue;
      depth[di] = z;
      putPixel(
        img, x, y,
        w.a * ca[0] + w.b * cb[0] + w.g * cc[0],
        w.a * ca[1] + w.b * cb[1] + w.g * cc[1],
        w.a * ca[2] + w.b * cb[2] + w.g * cc[2],
        255
      );
    }
  }
}

function cubeMesh() {
  const p = [
    [-1, -1, -1], [1, -1, -1], [1, 1, -1], [-1, 1, -1],
    [-1, -1, 1], [1, -1, 1], [1, 1, 1], [-1, 1, 1],
  ];
  const faces = [
    [0, 1, 2, 3, [0, 0, -1]],
    [5, 4, 7, 6, [0, 0, 1]],
    [4, 0, 3, 7, [-1, 0, 0]],
    [1, 5, 6, 2, [1, 0, 0]],
    [3, 2, 6, 7, [0, 1, 0]],
    [4, 5, 1, 0, [0, -1, 0]],
  ];
  const uv = [[0, 1], [1, 1], [1, 0], [0, 0]];
  const tris = [];
  faces.forEach((f) => {
    const n = { x: f[4][0], y: f[4][1], z: f[4][2] };
    const idx = [0, 1, 2, 0, 2, 3];
    for (let k = 0; k < 2; k++) {
      const i0 = f[idx[k * 3]], i1 = f[idx[k * 3 + 1]], i2 = f[idx[k * 3 + 2]];
      tris.push({

```

```

        p: [p[i0], p[i1], p[i2]],
        n,
        uv: [uv[idx[k * 3]], uv[idx[k * 3 + 1]], uv[idx[k * 3 + 2]]],
    });
}
});
return tris;
}

function lambert(n, l, kd, ka) {
    kd = kd == null ? 0.8 : kd;
    ka = ka == null ? 0.15 : ka;
    const d = Math.max(0, vdot(n, l));
    return ka + kd * d;
}

function blinnPhong(n, l, v, kd, ks, shin, ka) {
    kd = kd == null ? 0.65 : kd;
    ks = ks == null ? 0.35 : ks;
    shin = shin == null ? 32 : shin;
    ka = ka == null ? 0.12 : ka;
    const ndl = vdot(n, l);
    const diff = kd * Math.max(0, ndl);
    let spec = 0;
    if (ndl > 0) {
        const h = vnormalize(vadd(l, v));
        spec = ks * Math.pow(Math.max(0, vdot(n, h)), shin);
    }
    return ka + diff + spec;
}

function toSrgb(c) {
    return Math.pow(clamp(c, 0, 1), 1 / 2.2) * 255;
}

function fromSrgb(b) {
    return Math.pow(clamp(b / 255, 0, 1), 2.2);
}

function sampleNearest(tex, u, v, mode) {
    if (mode === "repeat") {
        u = u - Math.floor(u);
        v = v - Math.floor(v);
    } else {
        u = clamp(u, 0, 1);
        v = clamp(v, 0, 1);
    }
    const x = Math.min(tex.width - 1, Math.floor(u * tex.width));
    const y = Math.min(tex.height - 1, Math.floor(v * tex.height));
    const i = (y * tex.width + x) * 4;
    return [tex.data[i], tex.data[i + 1], tex.data[i + 2], tex.data[i + 3]];
}

function checkerTex(size) {
    size = size || 64;
    const data = new Uint8ClampedArray(size * size * 4);
    for (let y = 0; y < size; y++) {
        for (let x = 0; x < size; x++) {
            const on = ((x >> 3) + (y >> 3)) & 1;
            const i = (y * size + x) * 4;
            const c = on ? 230 : 40;
            data[i] = c;
            data[i + 1] = on ? 70 : 180;
            data[i + 2] = on ? 70 : 80;
            data[i + 3] = 255;
        }
    }
}

```

```

    return { width: size, height: size, data };
}

function projectTri(tri, PVM, width, height) {
    const pts = [];
    for (let i = 0; i < 3; i++) {
        const p = tri.p[i];
        const clip = m4mulVec(PVM, p[0], p[1], p[2], 1);
        if (clip.w <= EPS) return null;
        const ndc = toNdc(clip);
        const pix = viewport(ndc, width, height);
        pts.push({ x: pix.x, y: pix.y, z: pix.z, uv: tri.uv[i] });
    }
    return pts;
}

```

```

function drawWorldAxes(img, PVM, len) {
    len = len || 1.4;
    const o = m4mulVec(PVM, 0, 0, 0, 1);
    if (o.w <= EPS) return;
    const orig = viewport(toNdc(o), img.width, img.height);
    [
        [[len, 0, 0], [220, 60, 60]],
        [[0, len, 0], [60, 180, 80]],
        [[0, 0, len], [60, 100, 220]],
    ].forEach((ax) => {
        const q = m4mulVec(PVM, ax[0][0], ax[0][1], ax[0][2], 1);
        if (q.w <= EPS) return;
        const p = viewport(toNdc(q), img.width, img.height);
        const steps = 40;
        for (let i = 0; i <= steps; i++) {
            const t = i / steps;
            putPixel(
                img,
                orig.x + (p.x - orig.x) * t,
                orig.y + (p.y - orig.y) * t,
                ax[1][0], ax[1][1], ax[1][2]
            );
        }
    });
}

```

```

CG.vec3 = vec3;
CG.vadd = vadd;
CG.vsub = vsub;
CG.vscale = vscale;
CG.vdot = vdot;
CG.vcross = vcross;
CG.vlen = vlen;
CG.vnormalize = vnormalize;
CG.vlerp = vlerp;
CG.cross2 = cross2;
CG.m4ident = m4ident;
CG.m4mul = m4mul;
CG.m4mulVec = m4mulVec;
CG.m4translate = m4translate;
CG.m4scale = m4scale;
CG.m4rotateX = m4rotateX;
CG.m4rotateY = m4rotateY;
CG.m4rotateZ = m4rotateZ;
CG.m4lookAt = m4lookAt;
CG.m4perspective = m4perspective;
CG.m4ortho = m4ortho;

```

```
CG.m4normal = m4normal;
CG.putPixel = putPixel;
CG.clear = clear;
CG.over = over;
CG.overPixel = overPixel;
CG.barycentric = barycentric;
CG.fillTriangle = fillTriangle;
CG.viewport = viewport;
CG.toNdc = toNdc;
CG.makeDepth = makeDepth;
CG.fillTriangleDepth = fillTriangleDepth;
CG.cubeMesh = cubeMesh;
CG.lambert = lambert;
CG.blinnPhong = blinnPhong;
CG.toSrgb = toSrgb;
CG.fromSrgb = fromSrgb;
CG.sampleNearest = sampleNearest;
CG.checkerTex = checkerTex;
CG.projectTri = projectTri;
CG.drawWorldAxes = drawWorldAxes;
CG.clamp = clamp;

global.CG = CG;
})(typeof window !== "undefined" ? window : globalThis);
```

raster.js

```
/**
 * Tiny helper: ImageData software framebuffer + rAF loop.
 */
(function (global) {
  "use strict";

  const Raster = {};

  function boot(opts) {
    const canvas = document.getElementById(opts.canvas || "c");
    const out = document.getElementById(opts.out || "out");
    const ctx = canvas.getContext("2d", { alpha: false });
    const state = Object.assign({ t: 0, running: true }, opts.state || {});
    let img = ctx.createImageData(canvas.width, canvas.height);

    function frame(now) {
      if (!state._t0) state._t0 = now;
      state.t = (now - state._t0) / 1000;
      if (opts.draw) opts.draw(img, state);
      ctx.putImageData(img, 0, 0);
      if (opts.status && out) out.textContent = opts.status(state);
      if (state.running) requestAnimationFrame(frame);
    }
    requestAnimationFrame(frame);

    document.querySelectorAll("[data-toggle]").forEach((el) => {
      el.addEventListener("click", () => {
        const k = el.getAttribute("data-toggle");
        state[k] = !state[k];
      });
    });

    Raster._last = { canvas, ctx, img, state };
    return Raster._last;
  }

  Raster.boot = boot;
  global.Raster = Raster;
})(typeof window !== "undefined" ? window : globalThis);
```